

Übersetzung von Vorlesungsunterlagen in TypeScript TheoInf I + II

Projektarbeit

des Studiengangs Angewandte Informatik
an der Dualen Hochschule Baden-Württemberg Mannheim

von
Tobias Neithöfer und Leo Vatter

Abgabedatum: 15. April 2026

Bearbeitungszeitraum

01.10.2025 - 15.04.2026

Kurs

TINF23AI2

Gutachter

Prof. Dr. Karl Stroetmann

Unterschrift Gutachter

Erklärung

gemäß § 35 (1) der „Studien- und Prüfungsordnung DHBW Technik“ vom 7. März 2024.

Wir haben die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel verwendet.

Wir versichern zudem, dass die eingereichte elektronische Fassung mit der gedruckten Fassung übereinstimmt.*

* falls beide Fassungen gefordert sind

Mannheim, 14. April 2026

Ort, Datum

Unterschrift

Abstrakt

Diese Studienarbeit dokumentiert die systematische Portierung von rund 80 Jupyter-Notebooks der Lehrveranstaltungen „Theoretische Informatik I (Logik)“ und „Theoretische Informatik II (Algorithmen und Datenstrukturen)“ von Python nach TypeScript. Ziel der Arbeit war es zu evaluieren, ob sich durch die statische Typisierung von TypeScript die Codequalität und Nachvollziehbarkeit in der Lehre verbessern lässt, ohne die didaktische Zugänglichkeit einzuschränken.

Methodisch erfolgte die Übersetzung der Lehrmaterialien durch den systematischen Einsatz Künstliche Intelligenz (KI)-gestützter Tools (wie ChatGPT und Gemini) in Kombination mit manuellen Code-Optimierungen. Da TypeScript im Gegensatz zu Python bestimmte Konstrukte wie Set-Comprehensions, Operator Overloading oder die tiefgreifende wertbasierte Gleichheitsprüfung nicht nativ unterstützt, wurden funktionale Programmierparadigmen sowie spezialisierte Bibliotheken (z.B. `recursive-set`) erfolgreich in die Jupyter-Laufzeitumgebung integriert.

Die Ergebnisse zeigen, dass die Portierung erfolgreich abgeschlossen werden konnte und die Notebooks uneingeschränkt im Vorlesungsbetrieb einsetzbar sind. Die didaktische Evaluierung macht deutlich, dass das strikte Typsystem von TypeScript Studierende zwar zu einer präziseren Auseinandersetzung mit Signaturen und Datenstrukturen zwingt und die anfängliche Syntax etwas komplexer wirken lässt, dadurch aber das tiefere Verständnis nachhaltig fördert. Darüber hinaus belegen quantitative Laufzeitmessungen, dass die TypeScript-Implementierungen dank der JIT-Kompilierung der V8-Engine den interpretierten Python-Originalen bei rechenintensiven Algorithmen in der Performance signifikant überlegen sind.

Abstract

This paper documents the systematic porting of approximately 80 Jupyter notebooks for the courses „Theoretical Computer Science I (Logic)“ and „Theoretical Computer Science II (Algorithms and Data Structures)“ from Python to TypeScript. The aim of this work was to evaluate whether the static typing of TypeScript can improve code quality and comprehensibility in teaching without compromising didactic accessibility.

Methodologically, the translation of the teaching materials was carried out through the systematic use of Artificial Intelligence (AI)-assisted tools (such as ChatGPT and Gemini) in combination with manual code optimizations. Since TypeScript, unlike Python, does not natively support certain constructs such as set comprehensions, operator overloading, or deep value-based equality checking, functional programming paradigms and specialized libraries (e.g. `recursive-set`) were successfully integrated into the Jupyter runtime environment.

The results show that the porting was successfully completed and the notebooks are fully ready for use in lectures. The didactic evaluation reveals that while TypeScript’s strict type system requires students to engage more precisely with type signatures and data structures, making the initial syntax appear slightly more complex, it sustainably fosters a deeper understanding. Furthermore, quantitative runtime measurements demonstrate that the TypeScript implementations, thanks to the JIT compilation of the V8 engine, significantly outperform the interpreted Python originals in compute-intensive algorithms.

Inhaltsverzeichnis

Abbildungsverzeichnis	VI
Quellcodeverzeichnis	IX
Abkürzungsverzeichnis	X
1 Einleitung	1
1.1 Problemstellung	1
1.2 Zielsetzung	2
2 Grundlagen	4
2.1 Anaconda	4
2.2 Interaktive Programmierung mit Jupyter	5
2.3 TypeScript im Data-Science-Kontext	6
2.4 Setup und Toolchain	10
3 Einführung in TypeScript	13
3.1 Grundkonzepte: Typannotationen und Funktionen	14
3.2 Primitive Datentypen und Arithmetik	16
3.3 Ausgabe und Zeichenkettenlitterale	17
3.4 Arithmetische Operationen und Variablenzuweisung	19
3.5 Arrays	19
3.6 Tupel	25
3.7 Objekte	26
3.8 Boolesche Werte und Operatoren	27
3.9 Kontrollstrukturen	30
3.10 Numerische Funktionen	33
4 Methodische Vorgehensweise	35
4.1 Projektmanagement mit GitHub Projects	35
4.2 Der Übersetzungs-Workflow	36
4.3 Einsatz von KI-gestützten Tools	38

5	Case Study: Übersetzung des Notebooks 04-CNF	42
5.1	Semantische Grundlagen der Modellierung	42
5.2	Importe und Abhängigkeiten	43
5.3	Typdefinitionen	43
5.4	Eliminierung von $\leftrightarrow$	46
5.5	Negationsnormalform	49
5.6	Transformation in Konjunktive Normalform	51
5.7	Triviale Klauseln und Vereinfachung	53
5.8	Zusammenfassung der Übersetzungsmuster	55
6	Evaluierung und Ergebnisse	57
6.1	Lesbarkeit und syntaktischer Vergleich	57
6.2	Laufzeit und Performance	63
7	Fazit und Ausblick	70
7.1	Allgemeine Bewertung des Übersetzungsprozesses	70
7.2	Lesbarkeit und Didaktik	71
7.3	Performance-Betrachtung	71
7.4	Ausblick	72
	Literaturverzeichnis	XI

Abbildungsverzeichnis

1	GitHub Projects Kanban-Board - Stand 25.02.2026	35
---	---	----

Quellcodeverzeichnis

1	Dynamische Typisierung in Python	8
2	Statische Typisierung in TypeScript	9
3	Typhinweise in Python mit MyPy-Prüfung	9
4	Bereitstellung der Laufzeitumgebung	11
5	Integration des TypeScript-Kernels	11
6	Installation der Projekt-Abhängigkeiten	12
7	Typpannotationen bei Variablen	14
8	Variablendeklaration mit const und let	15
9	Funktionsdekларation mit Typpannotationen	15
10	Äquivalente Arrow-Funktion	15
11	Typüberprüfung numerischer Literale	16
12	Demonstration des Präzisionsverlusts bei Number	16
13	Verwendung von bigint	17
14	Ausgabe und Definition von Strings	18
15	Verwendung von Template Strings	18
16	Grundlegende arithmetische Operatoren	19
17	Ganzzahldivision und Modulo	19
18	Array mit numerischen Elementen	20
19	Zugriff auf Array-Elemente	20
20	Modifikation eines Array-Elements	20
21	Verkettung mit concat	21
22	Verkettung mit dem Spread-Operator	21
23	Bestimmung der Array-Länge	21
24	Filtern von Zahlen mit filter	21
25	Erzeugung einer Zahlenfolge mit Array.from	22
26	Verdoppeln von Zahlen mit map	23
27	Summierung mit reduce	24
28	Einfaches Array Destructuring	24
29	Destructuring mit Rest-Operator	25
30	Definition und Initialisierung eines Tupels	25
31	Zugriff auf Tupel-Elemente	25

32	Objektdefinition	26
33	Zugriffsarten auf Objekteigenschaften	26
34	Modifikation von Eigenschaften	27
35	Verschachtelte Objekte	27
36	Zugriff auf verschachtelte Daten	27
37	Logische Verknüpfungen	28
38	If-Else Verzweigung	30
39	Switch-Statement als Alternative	31
40	Klassische for-Schleife	31
41	for...of-Schleife	31
42	while-Schleife	32
43	Optimiertes Sieb des Eratosthenes	33
44	Importieren des Parsers und recursive-set	43
45	Python: Typalias-Set für Variablen, Formeln, Literale, Klauseln und Konjunktive Normalform (KNF)	44
46	TypeScript: Strukturell präzise Union-Typen und wertbasierte Mengen über recursive-set	44
47	Python: Literal ist typseitig zu allgemein für das Negationssymbol	44
48	TypeScript: Negierte Literale sind als Tupel mit festem Präfix ' $\neg$ ' typisiert	45
49	Python: Formula ist flexibel, aber typseitig kaum restriktiv	45
50	TypeScript: Formula erzwingt Operator-Menge und Stelligkeit über Union- Typen	46
51	Iteratives Type Narrowing der Formeln	46
52	Python: eliminateBiconditional mit Pattern Matching und einheitli- chem Formula-Typ	47
53	TypeScript: eliminateBiconditional mit Type Guards und Rückgabotyp Formula1	48
54	Eingeschränkter Union Type	49
55	Eingeschränkter Union Type von op	49
56	TypeScript: nnf mit Eingabetyp Formula2 und Rückgabotyp NNF	50
57	Python: nnf mit generischem Rückgabotyp Formula	51
58	Python: cnf mit Pattern Matching und nativen Sets	51
59	TypeScript: cnf mit Type Guards und RecursiveSet	52

60	Operator Overloading	52
61	TypeScript: <code>isTrivial</code> als Prädikat über <code>some</code> (funktionale Schreibweise) .	53
62	Python: <code>isTrivial</code> über <code>any(...)</code> und Generatorausdruck	54
63	TypeScript: <code>simplify</code> durch Filtern der Klauselmenge	54
64	Python: <code>simplify</code> als Set-Comprehension	54
65	Python-Beispiel: Potenzberechnung mit Typ-Annotationen	58
66	TypeScript-Beispiel: Potenzberechnung	58
67	Python-Beispiel: <code>shortest_path</code>	59
68	TypeScript-Beispiel: <code>shortestPath</code> mit nativem Set	60
69	Mengenvereinigung in Python	61
70	Mengenvereinigung in TypeScript mittels <code>.union()</code>	61
71	Imperative (KI-generierte) Überprüfung	62
72	Deklarative (optimierte) Überprüfung in TypeScript	62
73	Python-Referenz der Überprüfung	62
74	Time-Funktion in Python	64
75	Reduktionsschritt in Python mit Set-Comprehensions	66
76	Reduktionsschritt in TypeScript mit <code>RecursiveSet</code>	67
77	Reduktionsschritt in TypeScript mit <code>RecursiveSet</code>	68
78	Zeitmessung in Python (16-Damen)	68
79	Zeitmessung in TypeScript (16-Damen)	69

Abkürzungsverzeichnis

KI	Künstliche Intelligenz
AI	Artificial Intelligence
JSON	JavaScript Object Notation
KNF	Konjunktive Normalform
NNF	Negationsnormalform

1 Einleitung

Die vorliegende Studienarbeit dokumentiert die systematische Portierung von Jupyter-Notebooks aus der Programmiersprache Python nach TypeScript. Diese Notebooks bilden die Grundlage für die Lehrveranstaltungen „Theoretische Informatik I“ (Logik) und „Theoretische Informatik II“ (Algorithmen und Datenstrukturen) und dienen als interaktive Vorlesungsunterlagen zur Vermittlung theoretischer Konzepte der Informatik.

Die Verwendung von Jupyter-Notebooks in dem Informatikstudium hat sich in den letzten Jahren als effektive Methode etabliert, um theoretische Konzepte mit praktischen Implementierungen zu verbinden [3]. Während Python aufgrund seiner Verbreitung und einfachen Syntax häufig als erste Programmiersprache gewählt wird, stellt die dynamische Typisierung in komplexeren Projekten zunehmend eine Herausforderung dar. Diese Problematik führt zur zentralen Fragestellung dieser Arbeit: Lässt sich durch den Einsatz statischer Typisierung die Codequalität, Nachvollziehbarkeit und Effizienz in Lehrveranstaltungen verbessern, ohne dabei die didaktische Zugänglichkeit zu beeinträchtigen?

1.1 Problemstellung

Die aktuelle Implementierung der Vorlesungsunterlagen basiert auf 80 Jupyter-Notebooks. Prof. Dr. Karl Stroetmann formulierte die Aufgabenstellung wie folgt:

„Meine Vorlesungen Theoretische Informatik I+II [8] [7] verwenden eine Reihe von Jupyter-Notebooks, die auf der Sprache Python basieren. Die Sprache Python ist nur dynamisch getypt. Zwar gibt es für Python den Typ-Checker MyPy, aber das von MyPy unterstützte Typsystem ist wesentlich schwerfälliger als das Typsystem von TypeScript. Mein Ziel ist es daher, meine Vorlesungen von Python auf TypeScript umzustellen.“

Die zentrale Problematik ergibt sich aus mehreren Aspekten:

Mangelnde Typsicherheit in der informatischen Ausbildung

Python ist eine dynamisch typisierte Sprache, bei der Typfehler erst zur Laufzeit erkannt werden. Dies führt insbesondere in der Lehre zu Herausforderungen, da Studierende häufig erst spät, nach Ausführung des Codes, auf Typinkompatibilitäten

stoßen. Während Tools wie [MyPy](#) eine nachträgliche statische Typprüfung ermöglichen, bleibt die Integration in bestehenden Python-Projekte oft komplex und das Typsystem weniger ausdrucksstark als bei nativ statisch typisierten Sprachen [2]. Diese verzögerte Fehlerkennung kann das Verständnis für Typsysteme und deren Bedeutung in der Softwareentwicklung beeinträchtigen.

Eingeschränkte Möglichkeiten der statischen Codeanalyse

TypeScript bietet als Superset von JavaScript ein robustes, strukturelles Typsystem, das bereits während der Entwicklung umfassende Fehlerprüfungen ermöglicht [2]. Die statische Typprüfung zur Compile-Zeit erlaubt es, viele potenzielle Laufzeitfehler bereits im Vorfeld zu identifizieren und trägt somit zu einer verbesserten Code-Qualität bei. In der Lehre kann dies Studierenden ein tieferes Verständnis für Datenstrukturen und Algorithmen vermitteln, da Typen explizit deklariert und durch den Compiler überprüft werden.

Umfang des Portierungsprojekts

Mit insgesamt etwa 80 Notebooks stellt der Umfang der zu übersetzenden Materialien eine erhebliche Herausforderung dar. Diese Menge erfordert eine systematische Vorgehensweise sowie den strategischen Einsatz von [KI](#)-gestützten Übersetzungstools, um Konsistenz und Qualität über alle Notebooks hinweg zu gewährleisten.

1.2 Zielsetzung

Die vorliegende Arbeit verfolgt mehrere miteinander verbundene Zielsetzungen. Das vom Dozenten vorgegebene Hauptziel besteht in der vollständigen Übersetzung der 80 Python-basierten Jupyter-Notebooks nach TypeScript unter Verwendung von [KI](#)-Tools wie ChatGPT, Gemini, DeepSeek oder Grok. Diese Portierung soll die Vorlesungsmaterialien auf eine Sprache mit statischer Typisierung überführen und damit die didaktischen Möglichkeiten erweitern.

Ein zentrales Forschungsziel liegt in der Untersuchung, ob die in TypeScript implementierten Programme eine vergleichbare oder möglicherweise verbesserte Verständlichkeit aufweisen als die ursprünglichen Python-Implementierungen. Diese Fragestellung ist von besonderer Relevanz für die Lehrpraxis, da die Zugänglichkeit des Codes für Studierende

nicht durch die erhöhte Typsicherheit beeinträchtigt werden sollte.

Zusätzlich zur qualitativen Bewertung soll anhand der übersetzten Notebooks ein quantitativer Performanzvergleich zwischen Python und TypeScript durchgeführt werden. Dieser Vergleich kann Aufschluss über die praktische Eignung beider Sprachen für rechenintensive Algorithmen und Datenstrukturen geben. Die Arbeit folgt dabei einem systematischen Ansatz, der den Einsatz von [KI](#)-gestützten Übersetzungstools mit manueller Überprüfung und Optimierung kombiniert, wobei Best Practices für die TypeScript-Entwicklung berücksichtigt und die Übersetzungsqualität kontinuierlich überprüft werden.

2 Grundlagen

Dieses Kapitel erläutert die technologischen Grundlagen und Werkzeuge, die für die Durchführung dieser Studienarbeit relevant sind. Zunächst wird die Distributionsplattform Anaconda vorgestellt, gefolgt von einer detaillierten Betrachtung der interaktiven Entwicklungsumgebung Jupyter. Anschließend erfolgt eine Einführung in die Zielsprache TypeScript sowie der Vergleich mit Python. Den Abschluss bildet die Beschreibung der spezifischen Setup- und Toolchain-Konfiguration.

2.1 Anaconda

[Anaconda](#) ist eine weit verbreitete Open-Source-Distribution für die Programmiersprachen Python und R, die speziell für Aufgaben in den Bereichen Data Science, maschinelles Lernen und wissenschaftliches Rechnen entwickelt wurde. Sie vereinfacht das Paketmanagement und die Bereitstellung von Softwareumgebungen erheblich, was insbesondere in komplexen wissenschaftlichen Projekten von Vorteil ist. Kernkomponente der Distribution ist der Paketmanager [conda](#). Dieser ermöglicht es, Softwarepakete und deren Abhängigkeiten plattformübergreifend zu installieren, zu aktualisieren und zu verwalten. Ein wesentliches Merkmal von Anaconda ist die Fähigkeit, isolierte Umgebungen (Environments) zu erstellen. Diese Isolation verhindert Konflikte zwischen verschiedenen Versionen von Bibliotheken, die für unterschiedliche Projekte benötigt werden könnten.

Für die vorliegende Arbeit dient Anaconda als primäre Basisinfrastruktur. Obwohl das Ziel der Portierung auf TypeScript liegt, wird Anaconda ebenfalls verwendet, um die ursprünglichen Python-Notebooks auszuführen und zu validieren. Darüber hinaus fungiert das Anaconda Prompt (bzw. das Terminal) als Schnittstelle für systemweite Befehle. Hierüber erfolgen auch die notwendigen Installationen für das TypeScript-Ökosystem, wie beispielsweise die Ausführung von `npm install`, um [Node.js](#)-Pakete global oder lokal verfügbar zu machen. Die Verwaltung der Node.js-Laufzeitumgebung kann ebenfalls in die bestehende Infrastruktur integriert werden, was Anaconda zu einem vielseitigen Werkzeug macht.

2.2 Interaktive Programmierung mit Jupyter

Das [Jupyter Notebook](#) ist eine webbasierte, interaktive Entwicklungsumgebung, die es ermöglicht, Dokumente zu erstellen, die sowohl ausführbaren Code als auch formatierten Text, mathematische Formeln und Visualisierungen enthalten. Der Name „Jupyter“ ist ein Akronym, das sich ursprünglich aus den Kernsprachen Julia, Python und R ableitete, mittlerweile aber eine Vielzahl weiterer Programmiersprachen unterstützt.

Die Architektur von Jupyter basiert auf einem Client-Server-Modell, das die Trennung zwischen der Benutzeroberfläche und der Code-Ausführung ermöglicht.

- Das Frontend (Client): Der Benutzer interagiert über einen Webbrowser mit dem Notebook. Die Oberfläche erlaubt die Eingabe von Code in Zellen sowie die Gestaltung von Textbereichen mittels Markdown.
- Der Notebook-Server: Dieser Server verwaltet die Dateien und vermittelt die Kommunikation zwischen dem Frontend und dem ausführenden Kernel.
- Das Dateiformat (.ipynb): Jupyter Notebooks werden als JavaScript Object Notation ([JSON](#))-Dateien mit der Endung `.ipynb` gespeichert. Diese strukturierte Textdatei enthält den gesamten Inhalt des Notebooks, Code, Markdown-Text, Metadaten sowie die Ausgabeartefakte (z. B. Konsolenausgaben oder Grafiken), in einem standardisierten Format. Dies erleichtert die Versionierung und den Austausch von wissenschaftlichen Ergebnissen.

Ein zentrales Element der Jupyter-Architektur ist der Kernel. Der Kernel ist ein separater Prozess, der für die Ausführung des Codes zuständig ist, den der Benutzer im Notebook eingibt. Wenn eine Code-Zelle ausgeführt wird, sendet das Frontend den Code an den Kernel. Dieser führt die Anweisungen aus und sendet das Ergebnis zurück an das Frontend zur Darstellung. Für diese Studienarbeit ist das Kernel-Konzept von entscheidender Bedeutung:

Während die ursprünglichen Vorlesungsunterlagen auf dem Standard-Python-Kernel (ipykernel) basieren, erfordert die Portierung auf TypeScript die Integration eines dedizierten TypeScript-Kernels ([tslab](#)). Diese Architektur ermöglicht den Wechsel der Programmiersprache innerhalb derselben Benutzeroberfläche durch bloßen Austausch des Backends.

Die technische Integration des TypeScript-Kernels wird in Abschnitt [Setup und Toolchain](#) detailliert beschrieben.

2.3 TypeScript im Data-Science-Kontext

Während Python als De-facto-Standard im Bereich Data Science gilt, gewinnt das TypeScript-Ökosystem zunehmend an Bedeutung, insbesondere wenn Modelle direkt in Webanwendungen integriert oder visualisiert werden sollen.

2.3.1 Herausforderungen und Bibliotheks-Ökosystem

Eine wesentliche Herausforderung bei der Migration von Python zu TypeScript ist die Diskrepanz in der Standardbibliothek. Während Python mit Modulen wie [heapq](#) oder einer nativen Unterstützung für komplexe Mengenoperationen und Tupel ausgestattet ist, fehlen diese Konstrukte in der Standard-Laufzeitumgebung von TypeScript oft oder sind für wissenschaftliche Zwecke nicht performant genug implementiert. Zudem existiert zwar ein reiches Ökosystem ([npm](#)), dieses ist jedoch stark fragmentiert, im Gegensatz zu den monolithischen Standard-Paketen in Python wie beispielsweise NumPy.

Um die Funktionalität der Python-Vorlesungsunterlagen zu replizieren, wird in dieser Arbeit auf eine Auswahl spezialisierter Bibliotheken zurückgegriffen:

Mathematik und Logik:

Für numerische Stabilität und symbolische Berechnungen kommen [mathjs](#) (umfassende Mathematik-Bibliothek) und [fraction.js](#) (für rationale Zahlen zur Vermeidung von Fließkommafehlern) zum Einsatz. Für Constraint-Solving-Probleme und logische Beweise werden der [logic-solver](#) sowie der [z3-solver](#) (ein Theorem-Prover von Microsoft Research) eingebunden.

Datenstrukturen:

Da TypeScript keine native Priority Queue besitzt (äquivalent zu Pythons [heapq](#)), wird [heap-js](#) verwendet. Eine Besonderheit stellt die Bibliothek [recursive-set](#) dar. Native TypeScript Sets prüfen Objekte auf Referenzgleichheit, nicht auf Wertgleichheit. Das bedeutet, zwei inhaltlich identische Objekte mit gleicher Speicheradresse werden als unterschiedlich behandelt. Die Bibliothek [recursive-set](#) ermöglicht da-

her echte Mengenoperationen auf Basis der Deep Equality (rekursive Inhaltsgleichheit).

Visualisierung und Medien:

Zur Generierung von Graphen wird [@hpcc-js/wasm](#) genutzt. Dies ist eine WebAssembly-Portierung von Graphviz, die das Rendern von [DOT-Sprache](#) ermöglicht, ohne dass eine lokale `graphviz.exe`-Installation auf dem Host-System notwendig ist – ein entscheidender Vorteil für die Portabilität des Notebooks. Für die Bildmanipulation wird [pngjs](#) verwendet.

Type Definitions:

Ein Spezifikum der TypeScript-Entwicklung ist die Notwendigkeit separater Typ-Definitionen für Bibliotheken, die ursprünglich in reinem JavaScript geschrieben wurden. So muss beispielsweise für die Bildverarbeitung zusätzlich das Paket [@types/pngjs](#) als Dev-Dependency installiert werden, um die statische Typprüfung zu ermöglichen.

2.3.2 Syntaxvergleich und Typisierungskonzepte

Der fundamentale Unterschied zwischen Python und TypeScript liegt im Zeitpunkt und der Strenge der Typprüfung.

Statisch vs. Dynamisch / MyPy vs. TypeScript Compiler

Um diesen Unterschied zu verstehen, ist es hilfreich, zunächst zu klären, was ein Datentyp ist. Jeder Wert, den ein Programm verarbeitet, hat eine bestimmte Art: Eine ganze Zahl wie 42 verhält sich anders als ein Text wie "Hallo" oder ein Wahrheitswert wie True. Diese verschiedenen Arten von Werten heißen Datentypen. Der grundlegende Unterschied zwischen Python und TypeScript liegt darin, wann und ob überhaupt geprüft wird, ob diese Typen im Programm korrekt verwendet werden.

Dynamische Typisierung in Python

Python verwendet standardmäßig dynamische Typisierung. Das bedeutet, dass der Typ einer Variable nicht im Voraus festgelegt werden muss. Python ermittelt den Typ erst dann, wenn das Programm tatsächlich ausgeführt wird, also zur sogenannten Laufzeit. Als

Analogie lässt sich ein Rucksack vorstellen, in den beliebige Dinge (Datentypen) gepackt werden können: Mal liegt ein Buch darin, mal eine Flasche Wasser. Erst wenn etwas herausgenommen und festgestellt wird, dass es nicht passt, gibt es ein Problem.

In Python ist es daher ohne Weiteres möglich, einer Variable zunächst eine Zahl und später einen Text zuzuweisen:

```
1 x = 42          # x hat den Typ int (ganze Zahl)
2 x = "Hallo"     # x hat jetzt den Typ str (Text) Python erlaubt dies
3 # Der Fehler tritt erst beim Ausführen der nächsten Zeile auf:
4 x + 1
```

Listing 1: Dynamische Typisierung in Python

Den Fehler in der letzten Zeile würde Python erst bemerken, wenn diese Zeile tatsächlich ausgeführt wird. In einem großen Programm mit vielen Zeilen Code kann das dazu führen, dass Fehler lange unentdeckt bleiben, zum Beispiel dann, wenn eine bestimmte Stelle im Code nur selten erreicht wird.

Statische Typisierung in TypeScript

TypeScript hingegen verwendet statische Typisierung. Der Typ einer Variable muss entweder explizit angegeben werden oder wird vom Compiler automatisch erkannt. Diese automatische Erkennung heißt Typinferenz (abgeleitet von inferieren, also schlussfolgern). Entscheidend ist, dass TypeScript alle Typen bereits vor der Ausführung des Programms überprüft. Dieser Vorgang heißt Kompilierung, und das Programm, das diese Prüfung durchführt, heißt TypeScript-Compiler.

Der TypeScript-Compiler liest den gesamten Code durch und prüft, ob alle Typangaben widerspruchsfrei sind. Findet er einen Fehler, bricht er ab und gibt eine Fehlermeldung aus. Das Programm wird also gar nicht erst gestartet.

Das folgende Beispiel zeigt, wie TypeScript denselben Fehler verhindert. Eine Methode um in TypeScript eine neue Variable anzulegen, ist die Verwendung des Schlüsselworts `let`. Es signalisiert dem Compiler: Hier wird eine neue Variable erstellt. Dahinter folgt der Name der Variable, ein Doppelpunkt und der gewünschte Datentyp gefolgt von einem Gleichzeichen gefolgt von dem zuzuweisenden Wert.

```
1 let x: number = 42;
2 // Die folgende Zeile wuerde vom Compiler sofort rot markiert werden:
3 x = "Hallo";
4 // Fehlermeldung: Type 'string' is not assignable to type 'number'.
```

Listing 2: Statische Typisierung in TypeScript

Der Compiler würde bei der Zuweisung des Textes sofort einen Fehler melden, da in eine `number`-Variable kein `string` (Text) gelegt werden darf. Der Vorteil ist, dass solche Fehler bereits beim Schreiben des Codes und nicht erst beim Ausführen gefunden werden.

MyPy als optionale Typprüfung für Python

Da die dynamische Typisierung in größeren Projekten fehleranfällig sein kann, wurde für Python das Werkzeug MyPy entwickelt. MyPy ist ein statischer Typprüfer für Python: Es analysiert den Code auf Typfehler, indem es ihn lediglich durchliest, ohne das Programm tatsächlich zu starten. Anders als der TypeScript-Compiler, der den Code nach der Prüfung auch übersetzt, ist MyPy ein reines Kontrollwerkzeug und belässt den Python-Code unverändert.

Ein wesentlicher Unterschied zum TypeScript-Compiler ist, dass MyPy vollständig optional ist. Um MyPy nutzen zu können, müssen im Code freiwillige Typangaben vorhanden sein, sogenannte Type Hints (Typhinweise). Diese Typhinweise haben keinen Einfluss auf das Verhalten des Programms, Python ignoriert sie bei der Ausführung vollständig.

Das folgende Beispiel verdeutlicht dies mit einer einfachen Variable:

```
1 # Die Variable wird mit dem Typhinweis ': int' (ganze Zahl) versehen
2 alter: int = 20
3 # MyPy meldet hier VOR dem Start einen Fehler, da "Zwanzig" ein Text ist.
4 alter = "Zwanzig"
```

Listing 3: Typhinweise in Python mit MyPy-Prüfung

Bedeutung für die Übersetzung

Für die Übersetzung von den Python-Notebooks in TypeScript-Notebooks ergibt sich aus dem Unterschied der Typprüfung eine wichtige Herausforderung. Die Python-Notebooks können in drei Zuständen vorliegen:

1. **Vollständig ungetypter Code:** Es sind keine Typangaben vorhanden. Die Typen müssen also beim Übersetzen ermittelt und ergänzt werden.
2. **Teilweise getypter Code:** Nur manche Stellen enthalten Typhinweise. Hier müssen einerseits die fehlenden Typen für TypeScript ermittelt und ergänzt werden. Andererseits können die bereits vorhandenen Typangaben oft nicht einfach eins zu eins übernommen werden, sondern erfordern ebenfalls Anpassungen.
3. **Vollständig mit MyPy annotierter Code:** Alle relevanten Stellen enthalten Typhinweise. Diese können zwar oft direkt in TypeScript-Typen übertragen werden, jedoch müssen selbst hier häufig noch Präzisierungen vorgenommen werden. Der Grund dafür ist, dass das Typsystem von TypeScript in manchen Bereichen anders strukturiert oder strenger ist als das von MyPy.

2.4 Setup und Toolchain

Die Reproduzierbarkeit der Ergebnisse und die Stabilität der Entwicklungsumgebung sind essenziell für die Validierung der portierten Artefakte. Dieses Unterkapitel beschreibt die Architektur der verwendeten Toolchain und dokumentiert die notwendigen Konfigurationsschritte, wie sie in der [README.md](#) der Arbeit definiert sind.

2.4.1 Architektur der Laufzeitumgebung

Die technische Infrastruktur basiert auf einer hybriden Architektur, die das Python-zentrierte Ökosystem von Anaconda mit der JavaScript-basierten Node.js-Laufzeitumgebung verbindet. Diese Trennung ist notwendig, da Jupyter ursprünglich für Python konzipiert wurde, TypeScript jedoch eine Node.js-Laufzeit für die Transpilierung und Ausführung benötigt.

Die Toolchain gliedert sich in drei Schichten:

1. **Basisinfrastruktur (Anaconda):** Bereitstellung von Python, Jupyter (nbclassic) und Verwaltung der Systempfade.
2. **Laufzeitumgebung (Node.js):** Ausführungsumgebung für den TypeScript-Compiler und die genutzten Bibliotheken.

3. **Kernel-Layer (tslab):** Eine Middleware, die das Jupyter-Messaging-Protokoll implementiert und TypeScript-Codezellen an die Node.js-Laufzeit weiterleitet.

2.4.2 Implementierung der Umgebung

Die Einrichtung der Umgebung erfolgt deklarativ über die Kommandozeile der Anaconda-Distribution. Zunächst wird die Laufzeitumgebung durch die Installation von Node.js und dem klassischen Jupyter-Frontend vorbereitet. Dies schafft die Voraussetzungen für die Integration nicht-nativer Kernel.

Benötigte Programme:

1. Anaconda: <https://www.anaconda.com/download>
2. Node.js: <https://nodejs.org>

Danach erfolgt die Installation der Laufzeitumgebung über die Anaconda-Kommandozeile mit den folgenden Befehlen.

```
1  conda activate logic-algo
2  conda install nodejs
3  conda install -c conda-forge nbclassic
```

Listing 4: Bereitstellung der Laufzeitumgebung

Für die Ausführung von TypeScript innerhalb der Notebooks wird der Kernel `tslab` verwendet. Im Gegensatz zu reinen JavaScript-Kernen (wie `IJavascript`) ermöglicht `tslab` eine direkte Typ-Prüfung innerhalb der Zellen, was für die didaktische Zielsetzung der Arbeit, die Vermittlung von Typsicherheit, unabdingbar ist. Der `tslab`-Kernel und `typescript` werden über folgende Befehle bereitgestellt.

```
1  npm init -y
2  npm install -g tslab
3  tslab install
4  npm install typescript
```

Listing 5: Integration des TypeScript-Kernels

Die fachlichen Anforderungen der Vorlesungen erfordern spezifische Bibliotheken, die über den Paketmanager `npm` bezogen werden. Hierbei wird strikt zwischen Laufzeit-Abhängigkeiten (z. B. `mathjs` für Numerik, `recursive-set` für Mengenlehre) und Entwicklungs-Abhängigkeiten (z. B. `@types/pngjs`) unterschieden. Letztere sind notwendig, um für ursprünglich in JavaScript geschriebene Bibliotheken eine statische Typprüfung zu ermöglichen.

```
1 // Mathematische und logische Kernkomponenten
2 npm install fraction.js mathjs logic-solver z3-solver
3 // Datenstrukturen und Mengenoperationen
4 npm install heap-js recursive-set
5 // Visualisierung und Medienverarbeitung
6 npm install @hpcc-js/wasm pngjs
7 npm install --save-dev @types/pngjs
```

Listing 6: Installation der Projekt-Abhängigkeiten

2.4.3 Validierung der Toolchain

Um sicherzustellen, dass die Interaktion zwischen dem Jupyter-Frontend, dem `tslab`-Kernel und den installierten Node-Modulen korrekt funktioniert, wurde ein dediziertes Testverfahren etabliert. Das Referenz-Notebook [Test-Libraries.ipynb](#) führt einen systematischen Import-Test aller Komponenten durch. Dieses Notebook verifiziert, dass:

- der Kernel korrekt registriert ist und TypeScript-Syntax verarbeitet,
- alle externen Module im Pfad der Laufzeitumgebung gefunden werden,
- die Typdefinitionen korrekt aufgelöst werden.

Erst nach erfolgreicher Ausführung dieses Notebooks gilt die Umgebung als valide für die Bearbeitung der Vorlesungsunterlagen.

3 Einführung in TypeScript

TypeScript wurde von Microsoft entwickelt und erstmals im Oktober 2012 der Öffentlichkeit vorgestellt. Die Entwicklung stand unter der Leitung von Anders Hejlsberg, einem renommierten Softwarearchitekten, der zuvor bereits maßgeblich an der Gestaltung von C#, Delphi und Turbo Pascal beteiligt war. [11] [5] Der primäre Beweggrund für die Schaffung dieser neuen Sprache lag in den Defiziten von JavaScript bei der Entwicklung großer, komplexer Anwendungen [11].

Als Antwort auf diese Herausforderungen ist TypeScript als eine Erweiterung von JavaScript konzipiert, mit dem expliziten Ziel, die Entwicklung großer und langfristig wartbarer Anwendungen zu erleichtern. Syntaktisch ist TypeScript als Superset von ECMAScript definiert, sodass jedes gültige JavaScript-Programm zugleich ein ausführbares TypeScript-Programm ist. Zusätzlich zu JavaScript ergänzt TypeScript unter anderem Interfaces sowie ein reichhaltiges graduelles Typsystem, das eine schrittweise, inkrementelle Typisierung bestehender Codebasen unterstützt. [2]

Ein wesentliches Entwurfsprinzip ist dabei die *Typauslöschung*: Die Typinformationen werden vom Compiler zur statischen Analyse genutzt, sind jedoch nicht Bestandteil des erzeugten JavaScript-Codes, sodass zur Laufzeit keine TypeScript-Typen repräsentiert oder automatisch geprüft werden. Der TypeScript-Compiler überprüft TypeScript-Programme lediglich zur Übersetzungszeit und transformiert sie in JavaScript, wodurch sie unmittelbar in etablierten JavaScript-Laufzeitumgebungen ausgeführt werden können. Bierman, Abadi und Torgersen (2014) beschreiben TypeScript zudem als bewusst nicht vollständig statisch typischer ausgelegt, um gängige JavaScript-Idiome und die Typisierung existierender Bibliotheken praktisch zu unterstützen. [2]

Nach der Einführung in die Entstehungsgeschichte und die zentralen Entwurfsprinzipien von TypeScript soll im Folgenden die Programmiersprache selbst vorgestellt werden. Ziel ist es, einen Überblick über die grundlegenden Sprachkonstrukte zu geben, welche für die Vorlesungsunterlagen relevant sind.

Die Darstellung orientiert sich dabei am Notebook [Introduction.ipynb](#), das als Einstieg der Vorlesung *Theoretische Informatik* dient. Anhand dieses Notebooks werden zentrale Konzepte wie Variablendeklarationen, Typannotationen, Funktionen, Kontrollstrukturen, Arrays, Objekte sowie grundlegende Aspekte des Typsystems eingeführt und anhand von

Codebeispielen illustriert.

3.1 Grundkonzepte: Typannotationen und Funktionen

Bevor die einzelnen Datentypen und Sprachkonstrukte von TypeScript vorgestellt werden, führt dieser Abschnitt zwei Grundkonzepte ein, die im gesamten Kapitel wiederkehren: Typannotationen und Funktionen. Zudem werden die Schlüsselwörter `const` und `let` zur Variablendeklaration eingeführt, auf die im weiteren Verlauf ausführlicher eingegangen wird (siehe Abschnitt 3.2).

Typannotationen

Ein zentrales Merkmal von TypeScript ist die Möglichkeit, Variablen, Parametern und Rückgabewerten explizit einen Datentyp zuzuweisen. Diese sogenannten *Typannotationen* werden mit einem Doppelpunkt (:) nach dem jeweiligen Bezeichner notiert:

```
1 let name: string = "Alice"; // Variable vom Typ string
2 let age: number = 25; // Variable vom Typ number
3 let active: boolean = true; // Variable vom Typ boolean
```

Listing 7: Typannotationen bei Variablen

TypeScript unterstützt darüber hinaus die sogenannte *Typinferenz*: Wird ein Wert direkt bei der Deklaration zugewiesen, leitet der Compiler den Typ in den meisten Fällen automatisch ab, sodass eine explizite Annotation optional ist. Beide Schreibweisen sind im Laufe dieses Kapitels anzutreffen.

Variablendeklaration mit `const` und `let`

Zur Deklaration von Variablen werden in TypeScript die Schlüsselwörter `const` (für unveränderliche Variablen) und `let` (für veränderliche Variablen) verwendet. Beide sind block-scoped, das heißt sie sind nur innerhalb des unmittelbaren Block-Kontexts (zum Beispiel einer Funktion, einer Schleife oder eines if-Blocks) gültig, in dem sie deklariert wurden. Das ältere Schlüsselwort `var` wird in modernem TypeScript nicht mehr empfohlen.

```
1  const pi = 3.14; // unveränderlich
2  pi = 3;          // Fehler: Assignment to constant variable.
3  let counter = 0; // veränderlich
4  counter = 1;     // erlaubt
```

Listing 8: Variablendeklaration mit const und let

Funktionen

Eine *Funktion* ist ein benannter, wiederverwendbarer Codeblock, der eine definierte Aufgabe ausführt und optional einen Wert zurückgibt. In TypeScript wird eine Funktion mit dem Schlüsselwort **function** deklariert. Die Typen der Parameter sowie der Rückgabotyp werden mit Annotationen versehen. Der Rückgabotyp nach der Parameterliste legt fest, welchen Typ der zurückgegebene Wert haben muss. Das Schlüsselwort **return** beendet die Ausführung der Funktion und gibt den berechneten Wert an die aufrufende Stelle zurück. Gibt eine Funktion keinen Wert zurück, wird **void** als Rückgabotyp angegeben. Im folgenden Codeblock wird eine Funktion mit dem Namen *add* definiert, welche zwei Parameter *a* und *b* vom Type *number* entgegennimmt und *number* als Rückgabewert hat:

```
1  function add(a: number, b: number): number {
2      return a + b;
3  }
4  // Die Funktion wird wie folgt aufgerufen:
5  add(3, 5); // Ergebnis: 8
```

Listing 9: Funktionsdeklaration mit Typannotationen

Arrow-Funktionen

Neben der klassischen Syntax existiert eine kompaktere Schreibweise, die *Arrow-Function-Syntax* (Pfeilnotation). Das Schlüsselwort **function** entfällt; links des Pfeils (**=>**) stehen die Parameter, rechts der Rückgabedruck:

```
1  const add = (a: number, b: number): number => a + b;
2  add(3, 5); // Ergebnis: 8
```

Listing 10: Äquivalente Arrow-Funktion

Arrow-Funktionen werden in TypeScript häufig für kurze, einzeilige Ausdrücke verwendet und finden besonders in Kombination mit Array-Methoden wie `filter`, `map` und `reduce` Verwendung, die in Abschnitt 3.5 vorgestellt werden.

3.2 Primitive Datentypen und Arithmetik

In TypeScript, wie auch in JavaScript, werden alle numerischen Werte standardmäßig als Gleitkommazahlen gemäß dem [IEEE-754-Standard](#) repräsentiert. Es existiert kein separater Datentyp für Integer (Ganzzahlen). Dies gilt selbst dann, wenn die Darstellung eine Ganzzahl vermuten lässt. Der Operator `typeof` liefert zur Laufzeit den Typ seines Operanden als Zeichenkette und wird zur einfachen Typinspektion eingesetzt.

```
1  typeof 666    // Ausgabe: "number"
2  typeof 666.0  // Ausgabe: "number"
```

Listing 11: Typüberprüfung numerischer Literale

Diese Designentscheidung hat direkte Auswirkungen auf die Präzision arithmetischer Operationen. Die Genauigkeit für Ganzzahlen ist durch die Konstante `Number.MAX_SAFE_INTEGER` begrenzt, welche den Wert $2^{53} - 1$ (ca. 9 Milliarden) repräsentiert. Jenseits dieser Grenze ist die exakte Darstellung von Ganzzahlen nicht mehr gewährleistet, was zu Präzisionsverlusten führt.

```
1  Number.MAX_SAFE_INTEGER + 1 == Number.MAX_SAFE_INTEGER + 2
2  // Ausgabe: true
```

Listing 12: Demonstration des Präzisionsverlusts bei Number

Um die Limitierungen des `number`-Typs zu umgehen, existiert der primitive Datentyp `bigint`. Er ermöglicht die Darstellung von Ganzzahlen beliebiger Größe, beschränkt lediglich durch den verfügbaren Arbeitsspeicher. Syntaktisch werden `bigint`-Literele durch das Suffix `n` gekennzeichnet. Ergänzend steht der Wrapper-Konstruktor `BigInt()` zur Verfügung, der einen gegebenen Wert in den primitiven Typ `bigint` konvertiert, analog zur Beziehung zwischen `number` und dem Wrapper-Objekt `Number`.

Ein wichtiger Aspekt der Typsicherheit in TypeScript ist die strikte Trennung dieser

beiden numerischen Typen. Es ist nicht möglich, `bigint` und `number` in arithmetischen Operationen ohne explizite Konvertierung zu mischen. Zudem unterscheidet sich das Verhalten bei der Division: Während `number` stets Gleitkommaergebnisse liefert, führt die Division von `bigint`-Werten zu einer Trunkierung des Dezimalanteils (Ganzzahldivision).

```
1  const result: bigint = 7n ** 125n;  
2  // Berechnung extrem großer Potenzen  
3  5n / 2n  
4  // Ausgabe: 2n (Nachkommastellen werden abgeschnitten)
```

Listing 13: Verwendung von `bigint`

3.3 Ausgabe und Zeichenkettenlitterale

Für die Ausgabe von Werten auf der Konsole stellt TypeScript die Funktion `console.log` bereit. Diese Funktion ist polymorph (vielgestaltig) und akzeptiert Argumente beliebigen Typs, um sie in einer textuellen Repräsentation auszugeben. Ein elementarer Datentyp für textuelle Daten ist der `string`. In TypeScript können Zeichenkettenlitterale äquivalent durch einfache Anführungszeichen (') oder doppelte Anführungszeichen (") definiert werden. Es gibt semantisch keinen Unterschied zwischen diesen beiden Notationen, was Entwicklern Flexibilität bei der Einhaltung von Coding-Style-Guidelines gewährt. Zusätzlich existiert eine dritte Form von Zeichenkettenlitteralen, die durch Backticks (`) gekennzeichnet ist und als Grundlage für sogenannte Template Strings dient, auf die im Folgenden gesondert eingegangen wird.

In der interaktiven Umgebung eines Jupyter Notebooks wird zudem das Ergebnis des letzten Ausdrucks einer Zelle automatisch ausgegeben, ohne dass ein expliziter Aufruf von `console.log` notwendig ist.

```
1 // Explizite Ausgabe über die Konsole
2 console.log('Hello, World!');
3
4 // Alternative Syntax mit doppelten Anführungszeichen
5 console.log("Hello, World!")
6
7 // Implizite Ausgabe des letzten Ausdrucks einer Codezelle
8 'Hello, World!'
```

Listing 14: Ausgabe und Definition von Strings

Template String

Zusätzlich zu den klassischen Zeichenkettenliteralen unterstützt TypeScript sogenannte *Template Strings*. Diese werden durch Backticks (``) umschlossen und bieten erweiterte Funktionalitäten gegenüber herkömmlichen Strings. Ein zentrales Merkmal ist die String-Interpolation, die es ermöglicht, Ausdrücke direkt in eine Zeichenkette einzubetten. Hierbei wird die Syntax `\${...}` verwendet, wobei der Inhalt der geschweiften Klammern ausgewertet und das Ergebnis in einen *string* konvertiert wird. Dies erhöht die Lesbarkeit des Codes erheblich, da komplexe Verkettungsoperationen vermieden werden. Zudem können Template Strings über mehrere Zeilen gehen, ohne dass man dafür spezielle Zeichen wie `\n` im Code einfügen muss.

```
1 const name = "Alice";
2 const age = 25;
3
4 // Direkte Einbettung von Variablen
5 console.log(`User ${name} is ${age} years old.`);
6 // Ausgabe: User Alice is 25 years old.
7
8 // Auswertung von Ausdrücken innerhalb des Strings
9 console.log(`Next year, she will be ${age + 1}.`);
10 // Ausgabe: Next year, she will be 26.
```

Listing 15: Verwendung von Template Strings

3.4 Arithmetische Operationen und Variablenzuweisung

TypeScript stellt, analog zu JavaScript, die üblichen arithmetischen Operatoren zur Verfügung. Hierzu zählen die Addition (+), Subtraktion (-), Multiplikation (*), Division (/) sowie der Exponentiationsoperator (**) und der Modulo-Operator (%), der den Rest einer Division bestimmt.

```
1 1 + 2;    // Addition:      ergibt 3
2 5 - 3;    // Subtraktion:   ergibt 2
3 4 * 3;    // Multiplikation: ergibt 12
4 7 / 2;    // Division:      ergibt 3.5
5 7 % 3;    // Modulo:        ergibt 1
6 2 ** 10;  // Potenzierung:   ergibt 1024
```

Listing 16: Grundlegende arithmetische Operatoren

Ein wesentlicher Unterschied zu vielen anderen Programmiersprachen betrifft die Division. In TypeScript liefert der Standard-Divisionsoperator (/) stets eine Gleitkommazahl als Ergebnis, auch wenn beide Operanden Ganzzahlen sind. Um eine Ganzzahldivision (*Integer Division* oder Euklidische Division) durchzuführen, muss das Ergebnis explizit abgerundet werden. Hierfür steht die Funktion `Math.floor()` zur Verfügung, welche die größte Ganzzahl zurückgibt, die kleiner oder gleich dem übergebenen Argument ist, allerdings wird hierauf im Abschnitt 3.10 noch genauer eingegangen.

Der Modulo-Operator % berechnet den Rest dieser Division.

```
1 7 / 3;          // Standard-Division: ergibt 2.333...
2 Math.floor(7 / 3); // Ganzzahldivision: ergibt 2
3 7 % 3;          // Rest der Division: ergibt 1
```

Listing 17: Ganzzahldivision und Modulo

3.5 Arrays

Arrays sind eine fundamentale Datenstruktur in TypeScript, die eine Liste von Elementen repräsentiert. Sie entsprechen konzeptionell den Listen in Python, unterscheiden sich jedoch in Syntax und bestimmten Verhaltensweisen. Ein Array wird in TypeScript durch

eckige Klammern `[` und `]` definiert. Das leere Array, also ein Array ohne Elemente, wird wie folgt notiert: `[]`

Ein Array mit konkreten Elementen wird durch Komma-separierte Werte innerhalb der eckigen Klammern erzeugt. Leerzeichen nach den Kommas sind optional, werden jedoch zur besseren Lesbarkeit empfohlen.

```
1 [1, 2, 3]
```

Listing 18: Array mit numerischen Elementen

Zugriff auf Array-Elemente: Der Index-Operator

Der *Index-Operator* `[·]` ist ein Postfix-Operator, der verwendet wird, um auf das Element an einer bestimmten Position (Index) zuzugreifen. TypeScript verwendet, wie die meisten modernen Programmiersprachen, eine nullbasierte Indizierung, das heißt, das erste Element eines Arrays hat den Index 0. Es ist zu beachten, dass `const` lediglich die Neuzuweisung der Variable verhindert, nicht jedoch die Mutation des Arrays selbst. Die Elemente des Arrays können daher weiterhin verändert werden:

```
1 const arr = [1, 2, 3];
2 arr[0]; // Gibt das erste Element zurück: 1
```

Listing 19: Zugriff auf Array-Elemente

Der Index-Operator kann auch auf der linken Seite einer Zuweisung verwendet werden, um ein bestehendes Element zu modifizieren:

```
1 arr[0] = 7;
2 arr; // Ergebnis: [7, 2, 3]
```

Listing 20: Modifikation eines Array-Elements

Verkettung von Arrays

Arrays können mittels der Methode `concat` verkettet werden. Diese Methode erstellt ein neues Array, das alle Elemente des aufrufenden Arrays gefolgt von allen Elementen des als Argument übergebenen Arrays enthält.

```
1 [1, 2, 3].concat([4, 5, 6]); // Ergebnis: [1, 2, 3, 4, 5, 6]
```

Listing 21: Verkettung mit concat

Alternativ kann der *Spread-Operator* ... verwendet werden, um Arrays zu entpacken und zu kombinieren. Diese Syntax ist oft kompakter wenn mehrere Arrays konkateniert werden und wird in TypeScript häufig bevorzugt.

```
1 [...[1, 2, 3], ...[4, 5, 6]]; // Ergebnis: [1, 2, 3, 4, 5, 6]
```

Listing 22: Verkettung mit dem Spread-Operator

Die Eigenschaft `length` gibt die Anzahl der Elemente in einem Array zurück.

```
1 [4, 5, 4].length; // Ergebnis: 3
```

Listing 23: Bestimmung der Array-Länge

Die Methode `filter`

Die Methode `filter` erzeugt ein neues Array, das nur jene Elemente des ursprünglichen Arrays enthält, für die eine übergebene Bedingung erfüllt ist. Das ursprüngliche Array wird dabei nicht verändert. Die Bedingung wird als Funktion übergeben, die für jedes Element entweder `true` (Element wird behalten) oder `false` (Element wird verworfen) zurückgibt. In TypeScript wird für solche kurzen Funktionen häufig die *Arrow-Function-Syntax* (`=>`) verwendet.

Im folgenden Beispiel werden aus einer Liste von Zahlen nur jene herausgefiltert, die größer als 3 sind:

```
1 const numbers = [1, 2, 3, 4, 5];  
2 const result = numbers.filter(n => n > 3); // Ergebnis: [4, 5]
```

Listing 24: Filtern von Zahlen mit filter

Die Funktion `n => n > 3` wird intern für jedes Element des Arrays aufgerufen. Für `n = 1, 2` und `3` liefert sie `false`, weshalb diese Elemente verworfen werden. Für `n = 4` und `5` liefert sie `true`, sodass diese im Ergebnis-Array verbleiben. Das ursprüngliche Array `numbers` bleibt unverändert.

Element (n)	Bedingung (n > 3)	Ergebnis
1	false	Drop
2	false	Drop
3	false	Drop
4	true	Keep
5	true	Keep

Erzeugung von Arrays mit `Array.from`

Neben der literalen Schreibweise (mit eckigen Klammern `[]`) bietet TypeScript die statische Methode `Array.from`, um Arrays dynamisch zu erzeugen. Sie ist besonders nützlich, wenn ein Array einer bestimmten Länge mit berechneten Werten befüllt werden soll, beispielsweise um eine Zahlenfolge zu erzeugen.

Die Methode akzeptiert zwei Argumente: zunächst ein Objekt mit einer `length`-Eigenschaft, das die gewünschte Länge des Arrays festlegt, sowie eine optionale Mapping-Funktion, die für jeden Index aufgerufen wird und den Wert an dieser Position bestimmt. Eine solche Mapping-Funktion (Abbildungsfunktion) heißt so, weil sie jeden ursprünglichen Eingabewert systematisch auf einen neuen, veränderten Wert abbildet, indem sie ihn nach einer festgelegten Regel transformiert und dann als Ergebnis im neuen Array speichert. Im folgenden Beispiel wird ein Array mit den Zahlen 1 bis 10 erzeugt:

```

1 Array.from({length: 10}, (_, i) => i + 1);
2 // Ergebnis: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

```

Listing 25: Erzeugung einer Zahlenfolge mit `Array.from`

Die Argumente werden dabei wie folgt interpretiert:

1. `{length: 10}`: Ein Objekt, das `Array.from` signalisiert, ein Array der Länge 10 zu erzeugen. Da keine konkreten Werte angegeben sind, ist jeder Wert zunächst `undefined`.
2. `(_, i) => i + 1`: Die Mapping-Funktion bestimmt den tatsächlichen Wert an jeder Position. Sie erhält zwei Argumente: das Element selbst (hier `undefined`, da noch kein Wert vorhanden ist) und den aktuellen Index `i`. Da das Element nicht benötigt wird, ist es per Konvention mit einem Unterstrich (`_`) als *ignoriert* markiert. Der

Index `i` läuft von 0 bis 9 und wird durch `i + 1` in die gewünschten Werte 1 bis 10 transformiert.

Die Methode `map`

Die Methode `map` transformiert jedes Element eines Arrays mithilfe einer übergebenen Funktion und gibt ein neues Array zurück, das die transformierten Werte enthält. Die Länge des resultierenden Arrays ist stets identisch mit der des ursprünglichen Arrays.

Im folgenden Beispiel wird jedes Element einer Zahlenliste verdoppelt:

```
1  const numbers = [1, 2, 3, 4];  
2  const result = numbers.map(n => n * 2); // Ergebnis: [2, 4, 6, 8]
```

Listing 26: Verdoppeln von Zahlen mit `map`

Die Funktion `n => n * 2` wird für jedes Element einmal aufgerufen und gibt dessen doppelten Wert zurück. Das Ergebnis ist ein neues Array, in dem jede Position den transformierten Wert der entsprechenden Position im Original enthält. Das ursprüngliche Array `numbers` bleibt unverändert.

Die Methode `reduce`

Die Methode `reduce` verdichtet ein Array auf einen einzigen Wert, indem sie ein Element nach dem anderen mit einem laufenden Zwischenergebnis (dem sogenannten *Akkumulator*) verrechnet.

Der Aufruf `numbers.reduce((acc, n) => acc + n, 0)` setzt sich aus drei Teilen zusammen:

1. Der **Startwert**: Die 0 am Ende ist der initiale Wert für die Berechnung.
2. Die **Callback-Funktion**: `(acc, n) => acc + n` definiert die Rechenvorschrift.
3. Die **Parameter**:
 - `acc` (Abkürzung für *Accumulator*): Speichert das Zwischenergebnis aller bisherigen Additionen.
 - `n`: Ist das aktuelle Element aus dem Array, das gerade verarbeitet wird.

Im folgenden Beispiel wird die Summe aller Elemente eines Arrays berechnet:

```
1 const numbers = [1, 2, 3, 4];
2 const sum = numbers.reduce((acc, n) => acc + n, 0); // Ergebnis: 10
```

Listing 27: Summierung mit reduce

Der Akkumulator `acc` startet mit dem Wert 0. In jedem Schritt wird das aktuelle Element `n` zum bisherigen Akkumulatorwert addiert und das Ergebnis als neuer Akkumulatorwert weitergegeben:

Schritt	Akkumulator (acc)	Element (n)	Neuer acc
Start	0	—	0
1	0	1	1
2	1	2	3
3	3	3	6
4	6	4	10

Array Destructuring

Ein leistungsfähiges Feature von TypeScript ist das *Array Destructuring*. Es ermöglicht das Entpacken von Werten aus einem Array direkt in separate Variablen, anstatt mühsam über den Index (wie `arr[0]`, `arr[1]`) darauf zuzugreifen. Dies führt zu kompakterem und lesbarerem Code, insbesondere wenn Funktionen mehrere Werte in Form eines Arrays zurückgeben. TypeScript erlaubt die Deklaration einer Variablen mit `let` ohne sofortige Wertzuweisung. Der Wert kann zu einem späteren Zeitpunkt zugewiesen werden.

```
1 let a, b;
2 [a, b] = [10, 20];
3 console.log(a); // Ausgabe: 10
4 console.log(b); // Ausgabe: 20
```

Listing 28: Einfaches Array Destructuring

Destructuring kann auch mit dem *Rest-Operator* (`...`) kombiniert werden, um die verbleibenden Elemente eines Arrays in einer neuen Array-Variable zu sammeln. Dies ist besonders nützlich, um den *Head* und den *Tail* einer Liste zu trennen, ein gängiges Muster in der funktionalen Programmierung.

```
1 let head, tail;
2 [head, ...tail] = [10, 20, 30, 40, 50];
3 console.log(head); // Ausgabe: [10]
4 console.log(tail); // Ausgabe: [20, 30, 40, 50]
```

Listing 29: Destructuring mit Rest-Operator

3.6 Tupel

Ein *Tupel* ist in TypeScript ein spezieller Array-Typ, der eine feste Anzahl von Elementen mit vorab definierten Datentypen besitzt. Im Gegensatz zu herkömmlichen Arrays, die meist homogen typisiert sind (z. B. `string[]`), erlauben Tupel die Speicherung einer heterogenen Sammlung von Werten in einer strikten Reihenfolge. Dieses Konzept eignet sich besonders, um zusammengehörige Daten zu gruppieren, ohne dafür eigens ein komplexes Objekt definieren zu müssen, beispielsweise für Koordinatenpaare `[x, y]` oder Rückgabewerte einer Funktion.

Der Typ `boolean` bezeichnet dabei einen Wahrheitswert und kann ausschließlich die beiden Werte `true` oder `false` annehmen. Diese Wahrheitswerte werden insbesondere in Bedingungen und Kontrollstrukturen verwendet. Eine ausführlichere Behandlung boolescher Werte und Operatoren erfolgt im Abschnitt 3.8.

Im folgenden Beispiel wird ein Tupel-Typ `userProfile` definiert, der exakt drei Elemente in der Reihenfolge `number`, `string` und `boolean` erwartet:

```
1 let userProfile: [number, string, boolean]; // Typ-Definition
2 userProfile = [101, "Alice", true];
3 // Zuweisung: Reihenfolge und Typen müssen exakt übereinstimmen
```

Listing 30: Definition und Initialisierung eines Tupels

Der Zugriff auf die Elemente erfolgt analog zu Arrays über den Index-Operator:

```
1 console.log("Name:", userProfile[1]); // Ausgabe: Name: Alice
```

Listing 31: Zugriff auf Tupel-Elemente

3.7 Objekte

In TypeScript sind *Objekte* das grundlegende Strukturkonzept zur Modellierung zusammengesetzter Daten. Ein Objekt fasst Werte als *Properties* (*Eigenschaften*) zusammen, die jeweils aus einem Schlüssel (Property-Name) und einem zugehörigen Wert bestehen. Viele Spezialkonstrukte der Sprache, insbesondere Arrays, basieren zur Laufzeit ebenfalls auf Objekten, stellen jedoch zusätzliche, listenartige Operationen bereit (z. B. `length` und Array-Methoden).

In TypeScript werden Objektliterale mit geschweiften Klammern `{}` definiert. Das nachstehende Beispiel zeigt ein `student`-Objekt mit grundlegenden Eigenschaften:

```
1  const student = {  
2      firstName: "Marvin",  
3      matriculationNumber: 123456,  
4      isEnrolled: true  
5  };
```

Listing 32: Objektdefinition

Zugriff auf Eigenschaften

Für den Zugriff auf Objekteigenschaften existieren zwei gängige Notationen:

- **Punkt-Notation** (`obj.key`): Geeignet, wenn der Property-Name statisch bekannt ist und ein gültiger Bezeichner ist (z. B. `firstName`).
- **Klammer-Notation** (`obj[expr]`): Verwendet einen Ausdruck `expr`, der zur Laufzeit ausgewertet wird (z. B. eine Variable). Diese Notation ist insbesondere dann erforderlich, wenn der Schlüssel nicht als fester Bezeichner vorliegt oder Sonderzeichen enthält.

```
1  console.log(student.firstName);           // Punkt-Notation  
2  const key = "matriculationNumber";  
3  console.log(student[key]);               // Klammer-Notation mit Variable
```

Listing 33: Zugriffsarten auf Objekteigenschaften

Objekte sind mutable (*veränderlich*), sodass Werte bestehender Eigenschaften nachträglich modifiziert werden können:

```
1 student.isEnrolled = false;
```

Listing 34: Modifikation von Eigenschaften

Verschachtelte Objekte

Objekte können verschachtelt werden, um komplexe Datenstrukturen abzubilden. Das Beispiel `university` demonstriert eine Hierarchie, in der die Eigenschaft `location` selbst ein Objekt ist und `departments` ein Array von Strings darstellt.

```
1 const university = {
2   name: "HTWG Konstanz",
3   location: {
4     street: "Alfred-Wachtel-Str. 8",
5     city: "Konstanz",
6     zipCode: 78462
7   },
8   departments: ["Computer Science", "Architecture", "Mechanical Engineering"],
9   isPublic: true
10 };
```

Listing 35: Verschachtelte Objekte

Der Zugriff auf tiefere Ebenen erfolgt durch Verkettung des Punkt-Operators beziehungsweise Kombination mit dem Array-Index:

```
1 // Zugriff auf verschachteltes Objekt
2 university.location.city; // Ausgabe: Konstanz
3
4 // Zugriff auf Array innerhalb eines Objekts
5 university.departments[1]; // Ausgabe: Architecture
```

Listing 36: Zugriff auf verschachtelte Daten

3.8 Boolesche Werte und Operatoren

Die Repräsentation von Wahrheitswerten in TypeScript erfolgt über den primitiven Datentyp `boolean`, der ausschließlich die Werte `true` und `false` annehmen kann. Diese

Werte sind zentral für die Steuerung des Programmflusses und für logische Entscheidungen.

Logische Operatoren und Lazy Evaluation

TypeScript unterstützt die klassischen logischen Verknüpfungen Konjunktion (AND), Disjunktion (OR) und Negation (NOT). Die formalen Signaturen dieser Operatoren sehen wie folgt aus:

- `&& (a: boolean, b: boolean) => boolean`
- `|| (a: boolean, b: boolean) => boolean`
- `! (a: boolean) => boolean`

Ein essenzielles Konzept bei der Auswertung von `&&` und `||` ist die *Lazy Evaluation* (in diesem Kontext oft als *Short-Circuit Evaluation* oder Kurzschlussauswertung bezeichnet). Das bedeutet, dass der zweite Operand nur dann ausgewertet wird, wenn es für das Endergebnis zwingend notwendig ist:

- Bei `A && B` wird `B` nicht mehr ausgewertet, wenn `A` bereits `false` ist (das Ergebnis steht ohnehin als `false` fest).
- Bei `A || B` wird `B` nicht mehr ausgewertet, wenn `A` bereits `true` ist (das Ergebnis steht ohnehin als `true` fest).

```
1 true && false; // false
2 true || false; // true
3 !true;        // false
```

Listing 37: Logische Verknüpfungen

Vergleichsoperatoren und Strikte Gleichheit

Zur Erzeugung von Wahrheitswerten werden häufig Vergleiche herangezogen. Ein zentrales Konzept in TypeScript ist die Unterscheidung zwischen *striker* und *loser* Gleichheit:

- **Strikte Gleichheit (`===` und `!==`):** Vergleicht sowohl den Wert als auch den Datentyp. `5 === "5"` ergibt `false`, da eine Zahl mit einem String verglichen wird. Dies ist der empfohlene Standard in TypeScript.

- **Lose Gleichheit (== und !=):** Führt vor dem Vergleich eine automatische Typumwandlung (*Typ-Koerzition*) durch. `5 == "5"` ergibt `true`, da der String `"5"` implizit in die Zahl `5` umgewandelt wird. Dies führt häufig zu unerwartetem Verhalten und sollte vermieden werden.

Zusätzlich stehen die klassischen Größenvergleiche (`<`, `>`, `<=`, `>=`) zur Verfügung.

Ein wichtiger Unterschied zu Python besteht in der Verkettung von Vergleichen. Ein mathematischer Ausdruck wie `3 > 2 > 1` wird in TypeScript syntaktisch akzeptiert, liefert aber ein falsches Ergebnis. Die Auswertung erfolgt strikt von links nach rechts: Zuerst wird `3 > 2` zu dem Boolean `true` evaluiert. Anschließend versucht TypeScript den Vergleich `true > 1` zu berechnen. Hierbei greift die erwähnte Typ-Koerzition, bei der `true` als die Zahl `1` interpretiert wird. Der finale Vergleich lautet somit `1 > 1`, was zu dem fehlerhaften Ergebnis `false` führt. Korrekte mathematische Bereichsprüfungen müssen daher explizit mit dem logischen UND verknüpft werden: `3 > 2 && 2 > 1`.

Quantoren auf Arrays

TypeScript bietet Methoden auf Arrays, die den logischen Quantoren aus der Prädikatenlogik entsprechen:

1. **Allquantor ($\forall$):** Die Methode `every` prüft, ob *alle* Elemente eines Arrays eine Bedingung erfüllen.

```
1  const arr = [2, 4, 0, 7];
2  arr.every(x => x % 2 === 0);
3  // Sind alle Zahlen gerade? -> false, da 7 ungerade ist
```

2. **Existenzquantor ($\exists$):** Die Methode `some` prüft, ob *mindestens ein* Element die Bedingung erfüllt.

```
1  arr.some(x => x % 2 !== 0); // Ist mindestens eine Zahl ungerade? -> true
```

Diese Methoden ermöglichen eine deklarative und mathematisch fundierte Ausdrucksweise für Mengenoperationen direkt im Code.

3.9 Kontrollstrukturen

Kontrollstrukturen dienen dazu, den Ablauf eines Programms zu steuern. TypeScript bietet hierfür klassische Konstrukte, die syntaktisch stark an C, Java oder C++ angelehnt sind.

Verzweigungen

Die grundlegende bedingte Ausführung erfolgt mittels `if`, `else if` und `else`. Im Gegensatz zu Python müssen die Bedingungen in runde Klammern `()` gesetzt werden. Die zugehörigen Anweisungsblöcke werden üblicherweise in geschweifte Klammern `{}` eingeschlossen. Zwar erlaubt die TypeScript-Syntax das Weglassen der geschweiften Klammern, sofern der Block aus exakt einer einzigen Anweisung besteht, aus Gründen der Lesbarkeit und Fehlervermeidung (etwa beim späteren Hinzufügen weiterer Anweisungen) gilt die explizite Klammerung jedoch als Best Practice.

```
1  let x = 17;
2  if (x < 0) {
3      console.log('The number is negative!');
4  } else if (x === 0) {
5      console.log('The number is zero. ');
6  } else {
7      console.log("It's more than one.");
8  }
```

Listing 38: If-Else Verzweigung

Alternativ bietet die `switch`-Anweisung eine strukturierte Möglichkeit, mehrere Bedingungen gegen einen Wert zu prüfen. Am Ende jedes `case`-Blocks wird das Schlüsselwort `break` verwendet, um das Verlassen der `switch`-Struktur zu erzwingen. Fehlt das `break`, „fällt“ das Programm in den nächsten `case` durch. Innerhalb von Schleifen kann hier theoretisch auch das Schlüsselwort `continue` stehen, um den aktuellen Schleifendurchlauf vorzeitig zu beenden und mit der nächsten Iteration fortzufahren.

Ein interessantes Muster in TypeScript ist das `switch(true)`-Konstrukt, das es erlaubt, boolesche Ausdrücke anstelle von statischen Werten in den `case`-Zweigen auszuwerten:

```
1  switch (true) {
2      case (x < 0):
3          console.log('The number is negative!');
4          break;
5      case (x === 0):
6          console.log('The number is zero. ');
7          break;
8      default:
9          console.log("It's more than one.");
10 }
```

Listing 39: Switch-Statement als Alternative

Schleifenkonstrukte

Zur Iteration stellt TypeScript verschiedene Schleifenkonstrukte bereit:

- Die klassische **for-Schleife** nutzt eine Zählvariable, eine Laufbedingung und eine Schrittweite. Sie bietet maximale Kontrolle über den Iterationsprozess.

```
1  const arr = [1, 2, 3];
2  for (let i = 0; i < arr.length; i++) {
3      console.log(i);
4  }
```

Listing 40: Klassische for-Schleife

- Die **for...of-Schleife** eignet sich ideal, um direkt über die Elemente eines Arrays oder anderer iterierbarer Objekte zu laufen. Sie ist kompakter und weniger fehleranfällig als die klassische indexbasierte for-Schleife.

```
1  const arr = [1, 2, 3];
2  for (const element of arr) {
3      console.log(element);
4  }
```

Listing 41: for...of-Schleife

- Die **while-Schleife** führt einen Codeblock solange aus, wie eine definierte Laufbedingung erfüllt ist.

```
1  const arr = [1, 2, 3];
2  let i = 0;
3  while (i < arr.length) {
4      console.log(arr[i]);
5      i++;
6  }
```

Listing 42: while-Schleife

Anwendungsbeispiel: [Sieb des Eratosthenes](#)

Ein klassischer Algorithmus zur Bestimmung aller Primzahlen bis zu einer Grenze n ist das Sieb des Eratosthenes. Die Implementierung in TypeScript demonstriert das Zusammenspiel von Arrays, der klassischen `for`- sowie der `while`-Schleife und bedingten Anweisungen:

1. Initialisierung eines Arrays `primes`, wobei `primes[i] = i` gilt.
2. Iteration über alle Zahlen i und j , um Vielfache zu eliminieren (d. h. auf 0 zu setzen).
3. Filterung der verbleibenden Zahlen $\neq 0$.

Eine optimierte Version nutzt zudem `Math.ceil(Math.sqrt(n))` ($\lceil \sqrt{n} \rceil$) als Obergrenze für die äußere Schleife und beginnt die innere Schleife bei $j = i$, um redundante Prüfungen zu vermeiden.

```

1  let n = 10000;
2  // Initialisiere Array mit Werten 0 bis n
3  let primes = ({length: n + 1}, (_, i) => i);
4  let limit = Math.ceil(Math.sqrt(n));
5  for (let i = 2; i <= limit; i++) {
6      // Wenn i bereits markiert (0) ist, überspringen
7      if (primes[i] === 0) continue;
8      let j = i;
9      // Streiche alle Vielfachen von i (beginnend bei i*i)
10     while (i * j <= n) {
11         primes[i * j] = 0;
12         j++;
13     }
14 }
15 // Filtere alle Nicht-Null-Werte (Primzahlen)
16 let result = primes.filter(p => p !== 0 && p >= 2);

```

Listing 43: Optimiertes Sieb des Eratosthenes

3.10 Numerische Funktionen

TypeScript stellt über das zugrundeliegende JavaScript eine umfassende Sammlung mathematischer Standardfunktionen und Konstanten bereit, die im globalen `Math`-Objekt gekapselt sind. Dieses Objekt fungiert als Namensraum für statische Methoden und Eigenschaften. Zu den elementaren Funktionen gehören:

- **Wurzelberechnung:** `Math.sqrt(x)` berechnet die Quadratwurzel $\sqrt{x}$.
- **Betragsfunktion:** `Math.abs(x)` liefert den absoluten Wert $|x|$.
- **Potenzierung:** `Math.pow(base, exponent)` oder der Exponentialoperator `**`.

Rundungsfunktionen

Für die Umwandlung von Gleitkommazahlen in Ganzzahlen stehen drei spezifische Methoden zur Verfügung:

1. `Math.floor(x)`: Die *Gaußklammer* (oder Abrundungsfunktion) $\lfloor x \rfloor$. Sie rundet stets

auf die nächstkleinere ganze Zahl ab.

$$\text{floor}(x) = \max\{n \in \mathbb{Z} \mid n \leq x\}$$

2. `Math.ceil(x)`: Die *Aufrundungsfunktion* (Ceiling-Funktion) $\lceil x \rceil$. Sie rundet stets auf die nächstgrößere ganze Zahl auf.

$$\text{ceil}(x) = \min\{n \in \mathbb{Z} \mid n \geq x\}$$

3. `Math.round(x)`: Kaufmännisches Runden zur nächsten ganzen Zahl. Hierbei wird ab .5 aufgerundet.

Trigonometrische Funktionen

Das `Math`-Objekt umfasst auch die Standard-Trigonometriefunktionen wie `Math.sin`, `Math.cos` und `Math.tan` sowie deren Arkusfunktionen (`asin`, `acos`, `atan`). Es ist zu beachten, dass diese Funktionen Winkel im Bogenmaß (Radiant) erwarten bzw. zurückgeben.

Exponential- und Logarithmusfunktionen

Die Exponentialfunktion e^x wird über `Math.exp(x)` berechnet. Das Inverse, der natürliche Logarithmus $\ln(x)$, wird in TypeScript als `Math.log(x)` bezeichnet.

Konstanten

Wichtige mathematische Konstanten sind direkt zugänglich:

- `Math.PI` für die Kreiszahl $\pi \approx 3.14159$
- `Math.E` für die Eulersche Zahl $e \approx 2.718$

4 Methodische Vorgehensweise

Dieses Kapitel dokumentiert den systematischen Übersetzungsprozess der Notebooks, angefangen beim grundlegenden Verständnis der Funktionen über die KI-gestützte Übersetzung bis hin zur Validierung der funktionsfähigen Endprodukte.

4.1 Projektmanagement mit GitHub Projects

Angeichts des erheblichen Umfangs von etwa 80 zu portierenden Jupyter Notebooks erfordert die Durchführung dieser Studienarbeit eine strukturierte Projektplanung und klare Aufgabenteilung. Um die Zusammenarbeit im Team zu gestalten und den Fortschritt transparent zu verfolgen, wird das integrierte Planungswerkzeug *GitHub Projects* eingesetzt. Zu Beginn des Projekts wurden alle Notebooks als einzelne Tickets im Backlog erfasst. Jedem Ticket wurde anschließend ein verantwortliches Teammitglied zugewiesen. Die visuelle Verwaltung dieser Aufgaben erfolgt über ein Kanban-Board, welches den Arbeitsfluss in verschiedene Phasen unterteilt (siehe Abbildung 1).

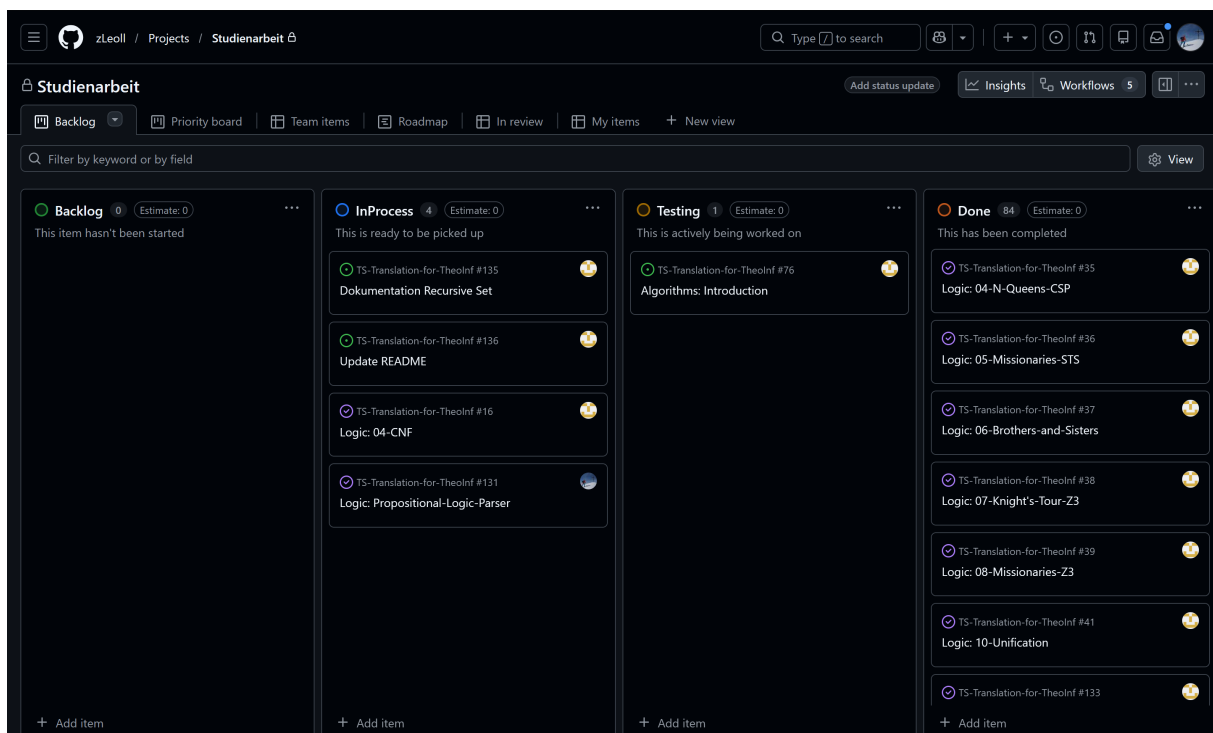


Abbildung 1: GitHub Projects Kanban-Board - Stand 25.02.2026

Das in Abbildung 1 dargestellte Board gliedert den Workflow in vier Spalten:

- **Backlog:** Sammelbecken für alle noch nicht begonnenen Aufgaben.
- **InProcess:** Tickets, die aktuell von einem Teammitglied bearbeitet werden.
- **Testing:** Notebooks, deren Übersetzung abgeschlossen ist und die zur Überprüfung bereitstehen.
- **Done:** Erfolgreich validierte und abgeschlossene Notebooks.

Der Übersetzungsprozess folgt einem definierten Zyklus: Sobald die Bearbeitung eines Notebooks beginnt, wird das entsprechende Ticket vom *Backlog* in den Status *InProcess* verschoben. Nach Abschluss der Portierung und dem Push der Änderungen in das Versionskontrollsystem wechselt das Ticket in den Status *Testing*. Dieser Status signalisiert, dass das Notebook für ein Code-Review bereitsteht. Gemäß dem Vier-Augen-Prinzip überprüft nun ein anderes Teammitglied die Implementierung auf Korrektheit und Funktionalität. Erst nach erfolgreicher Validierung und gegebenenfalls notwendigen Korrekturschleifen wird das Ticket final in die Spalte *Done* überführt. Dieser strukturierte Prozess gewährleistet eine kontinuierliche Qualitätssicherung über den gesamten Projektverlauf hinweg.

Eine ergänzende methodische Maßnahme zur Vorbereitung der schriftlichen Ausarbeitung stellt die Nutzung von Labels dar. Um relevante Erkenntnisse und Code-Beispiele bereits während der Implementierungsphase systematisch zu erfassen, wurde das Label *writable* eingeführt. Notebooks, die spezifische Herausforderungen oder diskussionswürdige Code-Fragmente enthalten, wurden mit diesem Label markiert. Diese Verschlagwortung ermöglicht eine Filterung der Tickets im Nachgang und gewährleistet, dass wichtige Aspekte der Portierung gezielt in die Dokumentation der Studienarbeit einfließen können, ohne dass eine erneute, vollständige Sichtung aller Notebooks notwendig ist.

4.2 Der Übersetzungs-Workflow

Die Portierung der Jupyter Notebooks von Python nach TypeScript folgt einem systematischen und iterativen Prozess, der manuelle Analysen mit KI-gestützter Automatisierung kombiniert. Dieser hybride Ansatz gewährleistet sowohl Effizienz bei der Code-

Konvertierung als auch die qualitative Sicherung der Inhalte. Der Workflow gliedert sich in die folgenden Phasen:

4.2.1 Analyse und Vorbereitung

Zu Beginn wird das zu bearbeitende Python-Notebook aus dem Repository des Dozenten herunter geladen und lokal ausgeführt. Dieser Schritt dient dazu, ein initiales Verständnis für die implementierten Algorithmen und deren Laufzeitverhalten zu gewinnen. Die Analyse der Python-Ausgabe bildet die Referenzbasis für die spätere Validierung der TypeScript-Implementierung.

4.2.2 Hybride Code-Übersetzung

Der Kernprozess der Übersetzung erfolgt semi-automatisch unter Einbeziehung von [KI-Modellen](#). Dabei werden einzelne Code-Snippets aus dem Python-Original extrahiert und mithilfe spezifischer Prompts, auf die im nachfolgenden Unterkapitel näher eingegangen wird, zur Übersetzung an das KI-Modell übergeben (siehe Abschnitt [4.3](#)). Die generierten TypeScript-Fragmente werden anschließend manuell überprüft und optimiert. Hierbei liegt der Fokus auf zwei Aspekten:

1. **Funktionale Äquivalenz:** Der übersetzte Code muss dasselbe Verhalten wie das Original zeigen. Abweichungen, die durch sprachspezifische Unterschiede entstehen (z.B. Kontrollstrukturen wie `switch-case` vs. `if-elif-else`), werden explizit geprüft und bei Bedarf angepasst.
2. **Syntaktische Nähe:** Gemäß den Projektvorgaben soll die TypeScript-Implementierung strukturell so nah wie möglich an der Python-Vorlage bleiben, um die Nachvollziehbarkeit zu erhalten.

Bei Diskrepanzen zwischen den Sprachkonzepten wird ergänzend die offizielle TypeScript-Dokumentation [\[4\]](#) konsultiert, um eine geeignete Lösung zu finden.

4.2.3 Integration und Test

Nach der Übersetzung aller Code-Zellen erfolgt ein umfassender Integrationstest des gesamten Notebooks. Hierbei wird verifiziert, ob die einzelnen Komponenten korrekt in-

teragieren und das erwartete Gesamtergebnis liefern. Fehlerquellen, die durch die statische Typisierung von TypeScript aufgedeckt werden, sowie Laufzeitfehler werden in dieser Phase korrigiert. Sollte die Einbindung externer Bibliotheken notwendig sein, wird der entsprechende Installationsbefehl (`npm install ...`) in der zentralen Projektdokumentation (`README.md`) ergänzt, um die Reproduzierbarkeit der Entwicklungsumgebung sicherzustellen.

4.2.4 Adaption der Texte

Erst nach erfolgreicher Validierung der Code-Basis erfolgt die Anpassung der begleitenden Markdown-Texte. Da sich durch den Sprachwechsel oft auch die Semantik (z. B. Variablendeklaration, Typisierung) ändert, müssen die Erläuterungen entsprechend aktualisiert werden. Zudem werden neu eingeführte Hilfsfunktionen, die in der Python-Version nicht existent waren, aber für die TypeScript-Umsetzung notwendig sind, dokumentiert eingebettet.

4.2.5 Qualitätssicherung und Review

Den Abschluss des Workflows bildet ein zweistufiger Review-Prozess. Zunächst führt der Bearbeiter eine finale Selbstkontrolle von Code und Text durch. Anschließend wird das Notebook auf einen dedizierten Feature-Branch im Versionskontrollsystem gepusht. Dies löst den Review-Prozess durch den zweiten Entwickler aus (Vier-Augen-Prinzip). Der Reviewer prüft die Änderungen auf Korrektheit, Stil und Verständlichkeit. Erst nach Freigabe und eventueller Überarbeitung erfolgt der Merge in den Hauptzweig (*Main-Branch*) sowie der Abschluss des zugehörigen Tickets im Projektmanagement-Tool.

4.3 Einsatz von KI-gestützten Tools

Ein zentraler Bestandteil der methodischen Vorgehensweise ist der Einsatz generativer KI-Modelle zur Unterstützung des Übersetzungsprozesses. Da die manuelle Portierung von rund 80 Jupyter Notebooks extrem zeitaufwendig und fehleranfällig wäre, wurden moderne Large Language Models als Assistenzsysteme in den Workflow integriert.

4.3.1 Prompt-Engineering-Strategien für die Code-Übersetzung

Um konsistente und syntaktisch korrekte Ergebnisse zu erzielen, wurde ein standardisierter System-Prompt entwickelt, der den KI-Modellen als Kontextrahmen dient. Es ist wichtig anzumerken, dass dieser Prompt keine finale, perfekt ausgereifte Lösung darstellte, sondern iterativ optimiert wurde. Nach der Übersetzung einzelner Notebooks und der Analyse der KI-Ergebnisse wurde der Prompt kontinuierlich angepasst und verfeinert, um wiederkehrende Fehler bei zukünftigen Portierungen zu minimieren.

Der verwendete Basis-Prompt adressierte spezifische Herausforderungen der Übersetzung von Python nach TypeScript, insbesondere im Hinblick auf das Typsystem und die modulare Struktur der Notebooks, und lautete wie folgt:

Das Jupyter Notebook, welches ich dir hochgeladen habe, soll von Python nach TypeScript übersetzt werden. Ich werde dir in den folgenden Nachrichten einzelne Zellen des Notebooks geben und diese übersetzt du dann bitte einzeln in TypeScript. Falls in der Zelle eine Klasse definiert wird, denke daran, dass TypeScript typgecheckt ist und dass es alle Methoden kennen muss. Definiere deshalb in der gleichen Zelle mit der Klasse zusammen ein Interface, welches alle Methoden und Attribute der Klasse enthält, die später im Python-Notebook noch der Klasse angehängt werden (z.B. bei einer Klasse Trie mit: `Trie.find = find`; `del find`). Um in späteren Zellen diese Methoden der jeweiligen Klasse noch anzuhängen, verwende `prototype` (z.B.: `Trie.prototype.find = function() ...`). Beachte auch immer den Kontext mit Hilfe der Markdown-Zellen, die vor den Code-Zellen stehen. Falls Graphviz in Python vorkommt, verwende „@hpcc-js/wasm“ und für die Ausgabe von HTML-Inhalten verwende `tslab` mit `display.html(...)`. Hier kommt die erste Zelle, die du übersetzen sollst: ...

Dieser Prompt wurde iterativ optimiert und adressiert mehrere kritische Aspekte:

- **Klassenstruktur und Typisierung:** Python erlaubt das dynamische Hinzufügen von Methoden zu Klassen zur Laufzeit (Monkey Patching [6]), was in den Algorithmen-Notebooks häufig genutzt wird. TypeScript als statisch typisierte Sprache verbietet dies standardmäßig. Der Prompt weist die KI explizit an, Interfaces und Prototypen-Manipulation zu nutzen, um diese Dynamik typsicher nachzubilden.
- **Bibliotheksmapping:** Spezifische Python-Bibliotheken wie Graphviz werden di-

rekt auf ihre TypeScript-Pendants (`@hpc-js/wasm`) gemappt, um Halluzinationen bei der Bibliothekswahl zu vermeiden.

- **Kernel-Spezifikation:** Die Anweisung zur Nutzung von `display.html(...)` stellt sicher, dass die Ausgabe in Notebooks, die Graphviz verwenden, im Notebook-Kontext korrekt gerendert wird.

Ergänzend wurde in vielen Interaktionen der Constraint

”Verwende keinen any-Type, keine as-Casts und keine non-null assertion Operatoren”

hinzugefügt, um die KI zur Generierung von typsicherem Code zu zwingen und die Verwendung von „Escape Hatches“ (Umgehung des Typsystems) zu minimieren.

4.3.2 Verwendete Modelle und Evolution

Im Verlauf des Projekts wurden verschiedene KI-Modelle evaluiert und eingesetzt. Zu Beginn erfolgte die Übersetzung primär mit ChatGPT (Modell GPT-4/4o, später dann GPT-5/5.1). Während die grundlegende Syntaxübersetzung zufriedenstellend war, zeigten sich Schwächen bei komplexeren Kontextabhängigkeiten, etwa bei der Referenzierung von Code über mehrere Notebook-Zellen hinweg. Dies führte dazu, dass nach subjektiver Einschätzung bei ca. 70 % der generierten Fragmente manuelle Nachbesserungen erforderlich waren.

Aufgrund dieser Einschränkungen erfolgte ein Wechsel zur Plattform Perplexity AI. Diese zeichnete sich durch eine präzisere Kontextverarbeitung und eine höhere Code-Qualität aus, was den Nachbearbeitungsaufwand auf subjektiv gesehen ca. 50 % reduzierte. Ein wesentlicher Vorteil von Perplexity ist die Fähigkeit, dynamisch das geeignetste Modell für eine Anfrage auszuwählen. Insbesondere der Einsatz des Modells *Gemini 3 Pro* in der experimentellen Phase erwies sich als besonders effektiv für die Übersetzung komplexer algorithmischer Logik.

4.3.3 Grenzen der KI-Unterstützung

Trotz des Einsatzes leistungsfähiger Modelle stieß die KI-Unterstützung an systembedingte Grenzen:

- **Verarbeitung ganzer Notebooks:** Erste Versuche, ein komplettes Notebook auf einmal hochzuladen und übersetzen zu lassen, scheiterten. Die KI stieß hierbei offensichtlich an praktische Grenzen, etwa bei Datei-Upload-Limits und Verarbeitungszeit, was zu abgebrochenen Anfragen oder fehlerhaften Ergebnissen führte. Gegen Ende der Übersetzungsarbeiten wurde erneut versucht, beispielsweise das Notebook [11-Prover.ipynb](#) vollständig zu übersetzen; aufgrund der hierfür erforderlichen Vielzahl an Dateiuploads wurde der Vorgang jedoch nach einigen Minuten mit einer Fehlermeldung abgebrochen oder mit dem Hinweis auf ein überschrittenes Upload-Limit beendet.
- **Fehleranalyse und Lernprozess:** Selbst bei abschnittsweiser Übersetzung produzierte die KI weiterhin Fehler, die zunächst verstanden und dann manuell korrigiert werden mussten. Vor diesem Hintergrund erwies sich die iterative Methode (Zelle für Zelle) als deutlich robusterer Ansatz. Das stückweise Vorgehen zwang nicht nur zur gründlichen Prüfung jedes Code-Snippets, sondern half auch dabei, sich tiefer in die algorithmischen Zusammenhänge der Thematik wieder einzuarbeiten, anstatt lediglich generierten Code ungeprüft zu übernehmen.
- **Halluzinationen bei Nischen-Bibliotheken:** Bei selten verwendeten TypeScript-Bibliotheken neigten die Modelle dazu, nicht existente Methoden zu erfinden oder APIs von ähnlichen JavaScript-Bibliotheken zu vermischen.

5 Case Study: Übersetzung des Notebooks 04-CNF

Das vorliegende Notebook [04-CNF.ipynb](#) demonstriert die Überführung von aussagenlogischen Formeln in die [KNF](#). Anhand dieser Case Study werden die grundlegenden Unterschiede zwischen Python und TypeScript in Bezug auf Typisierung, Datenstrukturen und Kontrollfluss veranschaulicht.

5.1 Semantische Grundlagen der Modellierung

In der Aussagenlogik bilden atomare Aussagen, repräsentiert durch Variablen (z. B. p oder q), das Fundament. Sie können lediglich die Wahrheitswerte *wahr* ($\top$) oder *falsch* ($\perp$) annehmen. Eine Formel entsteht durch die rekursive Verknüpfung dieser atomaren Bestandteile mittels logischer Junktoren: der Negation ($\neg$, NICHT), der Konjunktion ($\wedge$, UND), der Disjunktion ($\vee$, ODER) oder der Implikation ($\rightarrow$, WENN-DANN).

Für die algorithmische Normalisierung sind spezialisierte Strukturen entscheidend: Ein Literal ist die kleinste Einheit und bezeichnet entweder eine Variable selbst (positives Literal wie z. B. p) oder deren Negation (negatives Literal wie z. B. $\neg q$). Eine Klausel fasst eine Menge solcher Literale zusammen, wobei diese logisch als Disjunktion (ODER-Verknüpfung) interpretiert werden. Die Klausel $\{p, \neg q\}$ entspricht also dem Ausdruck $p \vee \neg q$. Die [KNF](#) schließlich besteht aus einer Menge von Klauseln, die als Konjunktion (UND-Verknüpfung) verstanden wird. Ein Beispiel für eine Menge S von Klauseln mit den Variablen p , q und r ist $S = \{\{p, \neg q\}, \{q, r\}, \{\neg p, \neg r\}\}$. Diese Mengendarstellung entspricht der aussagenlogischen Formel: $(p \vee \neg q) \wedge (q \vee r) \wedge (\neg p \vee \neg r)$. Die gesamte Formel ist somit nur dann wahr, wenn jede einzelne Klausel für sich erfüllt ist. Für die Transformation in diese Mengendarstellung werden im Notebook zwei weitere Normalformen genutzt, die ebenfalls eine klar definierte Struktur besitzen. Zunächst wird die Negationsnormalform ([NNF](#)) gebildet: Dabei werden Negationen so weit nach innen geschoben, dass der Operator $\neg$ ausschließlich noch unmittelbar vor Variablen steht, während die Formel ansonsten nur aus $\wedge$, $\vee$ und Variablen sowie ggf. den Konstanten $\top$ und $\perp$ besteht. Anschaulich bedeutet dies beispielsweise, dass eine Negation über einer zusammengesetzten Teilformel aufgelöst wird, etwa $\neg(p \wedge q) \rightsquigarrow (\neg p \vee \neg q)$, sodass die Negation nicht mehr „über“ dem $\wedge$ -Knoten steht, sondern direkt an die Variablen gebunden wird. Aufbauend auf dieser strukturell bereinigten Zwischenform wird anschließend die

KNF konstruiert. Ein typisches Vorher-Nachher-Beispiel ist die Verteilung einer Disjunktion über eine Konjunktion: $(p \wedge q) \vee r \rightsquigarrow (p \vee r) \wedge (q \vee r)$, wodurch die Formel in eine Konjunktion von Disjunktionen überführt wird. In der finalen Datenstruktur wird diese **KNF** dann nicht mehr als Operatorbaum gespeichert, sondern unmittelbar als Menge von Mengen, z. B. $(p \vee \neg q) \wedge (q \vee r) \rightsquigarrow \{\{p, \neg q\}, \{q, r\}\}$, wobei die äußere Menge die Konjunktion der Klauseln und jede innere Menge die Disjunktion der Literale repräsentiert.

5.2 Importe und Abhängigkeiten

Zunächst werden im TypeScript Notebook der `LogicParser` sowie spezielle Datenstrukturen aus der Bibliothek `recursive-set` importiert:

```

1  import { LogicParser } from './PropositionalLogicParser';
2  import { RecursiveSet, Tuple } from 'recursive-set';

```

Listing 44: Importieren des Parsers und recursive-set

Ein Parser ist in diesem Kontext zwingend erforderlich, da die aussagenlogischen Formeln zunächst als einfache Strings vorliegen (bspw. `'(p ∧ q) → r'`) und erst in eine strukturierte, maschinenlesbare Form, wie einen abstrakten Syntaxbaum, überführt werden müssen, bevor die anschließenden algorithmischen Transformationen zur **KNF** auf sie angewendet werden können. Die Python-Version bindet den Parser komfortabel per `%run Propositional-Logic-Parser.ipynb` ein ohne zusätzliche Bibliothek. Dies markiert einen der fundamentalsten Unterschiede zur Python-Welt: Während Python für herkömmliche Tupel, Mengen und unveränderliche Mengen (`frozenset`) standardmäßig eine inhaltliche Wertgleichheit überprüft, vergleicht TypeScript Objekte und Arrays lediglich über ihre Speicherreferenz. Um das Verhalten von Pythons `set`, `frozenset` und `tuple` in TypeScript nachzubilden, muss zwingend auf die Bibliothek `recursive-set` zurückgegriffen werden, welche die Klassen `RecursiveSet` und `Tuple` für genau diesen Zweck bereitstellt.

5.3 Typdefinitionen

Die Typdefinitionen in beiden Implementierungen verfolgen das gleiche fachliche Ziel (Formel, Literal, Klausel, **KNF**), unterscheiden sich jedoch grundlegend in ihrer *Präzision* und

damit in der Fähigkeit, fehlerhafte Strukturen frühzeitig auszuschließen. In Python dienen die Annotationen primär der Dokumentation, während TypeScript die zulässige Struktur des Syntaxbaums und der Mengenkonstrukte deutlich enger über Union-Typen und fest geformte Tupel beschreibt:

```

1 Variable = str
2 Formula = TypeVar('Formula')
3 Formula = Variable | tuple[Formula, ...]
4 Literal = Variable | tuple[str, Variable]
5 Clause = frozenset[Literal]
6 CNF = set[Clause]

```

Listing 45: Python: Typalias-Set für Variablen, Formeln, Literale, Klauseln und CNF

```

1 type Variable = string;
2 type Formula = Variable | ['T' | '⊥'] | ['¬', Formula] |
3   ['↔' | '→' | '∧' | '∨', Formula, Formula];
4 type Literal = Variable | Tuple<['¬', Variable]>;
5 type Clause = RecursiveSet<Literal>;
6 type CNF = RecursiveSet<Clause>;

```

Listing 46: TypeScript: Strukturell präzise Union-Typen und wertbasierte Mengen über recursive-set

In Python ist ein Literal entweder eine Variable (`str`) oder ein Tupel `tuple[str, Variable]`. Diese Definition ist bewusst knapp, aber sie lässt rein vom Typ her auch ungültige Konstruktionen zu, da das erste Tupel-Element nur als `str` und nicht als festes Symbol '¬' beschrieben ist.

```

1 l1: Literal = 'p'           # positives Literal
2 l2: Literal = ('¬', 'p')    # negatives Literal
3 # Typseitig ebenfalls möglich, semantisch jedoch falsch:
4 l3: Literal = ('not', 'p')
5 l4: Literal = ('∧', 'p')

```

Listing 47: Python: Literal ist typseitig zu allgemein für das Negationssymbol

In TypeScript wird das negative Literal als `Tuple<'¬', Variable>` modelliert. Ein Ausdruck der Form `X<...>` ist eine Parametrisierung eines Typs. Dadurch ist festgelegt, dass das Tupel exakt die Form `['¬', Variable]` besitzt, wodurch falsche Operator-Tupel bereits auf Typ-Ebene verhindert werden.

```

1  const l1: Literal = 'p';
2  const l2: Literal = new Tuple('¬', 'p');
3  // Strukturell falsche Literale wären nicht mit dem Typ vereinbar:
4  const l3: Literal = new Tuple('not', 'p');

```

Listing 48: TypeScript: Negierte Literale sind als Tupel mit festem Präfix `'¬'` typisiert

Python nutzt `frozenset[Literal]` als unveränderliche Menge von Literalen für Klauseln und `set[Clause]` für die KNF als Menge von Klauseln. Die Unveränderlichkeit der Klauseln ist entscheidend, weil nur unveränderliche Objekte in Python als Elemente einer äußeren Menge (Set) auftreten können.

Ebenso ist in Python `Formula` sehr allgemein als Variable oder beliebiges Tupel rekursiver Formeln modelliert (`tuple[Formula, ...]`). Dadurch lässt sich zwar jede baumartige Repräsentation typisieren, jedoch werden weder die erlaubten Operatoren noch die korrekte Stelligkeit (z. B. binär für $\wedge$, $\vee$, $\rightarrow$, $\leftrightarrow$) durch den Typ erzwungen.

```

1  # Typisch gemeint:
2  f1: Formula = ('∧', 'p', 'q')
3  f2: Formula = ['¬', ['∨', 'p', 'q']];
4  # Wird vom Typ ebenfalls akzeptiert, ist aber strukturell fehlerhaft:
5  f2: Formula = ('∧', 'p')           # falsche Stelligkeit
6  f3: Formula = ('XOR', 'p', 'q')    # Operator nicht vorgesehen

```

Listing 49: Python: `Formula` ist flexibel, aber typseitig kaum restriktiv

TypeScript beschreibt `Formula` dagegen als Union aus exakt vier Formen: Variable, Wahrheitskonstante, Negation mit genau einem Operand, sowie binärer Operator mit genau zwei Operanden. Damit wird die Stelligkeit direkt im Typ festgelegt. Zudem sind die Operator-Symbole explizit enumeriert.

```

1  const f1: Formula = ['^', 'p', 'q'];           // gültig
2  const f2: Formula = ['¬', ['∨', 'p', 'q']];     // gültig
3  // Strukturelle Fehler würden hier bereits durch den Typ auffallen:
4  const f3: Formula = ['^', 'p'];                // falsche Stelligkeit
5  const f4: Formula = ['XOR', 'p', 'q'];          // Operator nicht erlaubt

```

Listing 50: TypeScript: Formula erzwingt Operator-Menge und Stelligkeit über Union-Typen

Ein besonderes Merkmal der TypeScript-Version ist das iterative Einschränken des Typensystems (Type Narrowing). Im folgenden werden spezifischere Formel-Typen deklariert, bei denen sukzessive Operatoren wie ' $\leftrightarrow$ ' und ' $\rightarrow$ ' aus dem Union-Typ entfernt werden. Der NNF-Typ geht noch weiter und beschränkt die Negation explizit auf atomare Literale, sodass nachfolgende Funktionen statisch wissen, dass keine verschachtelten NOT-Knoten mehr auftreten. Dies erhöht die statische Typsicherheit in den nachfolgenden Transformationsschritten signifikant, während Python für alle Zwischenzustände durchgehend denselben Formula-Typ verwendet.

```

1  type Formula1 = Variable | ['T' | '⊥'] | ['¬', Formula1] |
2    ['→' | '^' | '∨', Formula1, Formula1];
3  type Formula2 = Variable | ['T' | '⊥'] | ['¬', Formula2] |
4    ['^' | '∨', Formula2, Formula2];
5  type NNF      = Variable | ['T'] | ['⊥'] | ['¬', Variable] |
6    ['^' | '∨', NNF, NNF];

```

Listing 51: Iteratives Type Narrowing der Formeln

5.4 Eliminierung von $\leftrightarrow$

Die Funktionen `eliminateBiconditional` und `eliminateConditional` markieren den Übergang von einer allgemeinen Formelrepräsentation hin zu einer strukturell eingeschränkten Zwischenform, in der die Operatoren ' $\leftrightarrow$ ' und ' $\rightarrow$ ' nicht mehr auftreten. In der TypeScript-Version wird dieser Schritt nicht nur algorithmisch, sondern zusätzlich *typseitig* abgesichert: `eliminateBiconditional` liefert den engeren Typ `Formula1`.

```

1 def eliminateBiconditional(f: Formula) -> Formula:
2     'Eliminate the logical operator " $\leftrightarrow$ " from the formula f.'
3     match f:
4         case str(p):          # This case covers variables.
5             return p
6         case (' $\leftrightarrow$ ', g, h):
7             return eliminateBiconditional(('^', (' $\rightarrow$ ', g, h), (' $\rightarrow$ ', h, g)))
8         case ('T', ) | (' $\perp$ ', ):
9             return f
10        case (' $\neg$ ', g):
11            return (' $\neg$ ', eliminateBiconditional(g))
12        case (op, g, h):      # This case covers ' $\rightarrow$ ', '^', and ' $\vee$ '.
13            return (op, eliminateBiconditional(g), eliminateBiconditional(h))
14        return None # type: ignore

```

Listing 52: Python: `eliminateBiconditional` mit Pattern Matching und einheitlichem Formula-Typ

```

1  function eliminateBiconditional(f: Formula): Formula1 {
2      'Eliminate the logical operator "↔" from f.'
3      if (typeof f === 'string') {
4          return f;
5      }
6      const [op] = f;
7      switch (op) {
8          case '↔': {
9              const [, g, h] = f;
10             return eliminateBiconditional(['^', ['→', g, h], ['→', h, g]]);
11         }
12         case '⊤':
13             return ['⊤'];
14         case '⊥':
15             return ['⊥'];
16         case '¬': {
17             const [, g] = f;
18             return ['¬', eliminateBiconditional(g)];
19         }
20         case '→':
21         case '^':
22         case '∨': {
23             const [, g, h] = f;
24             return [op, eliminateBiconditional(g), eliminateBiconditional(h)];
25         }
26     }
27 }

```

Listing 53: TypeScript: `eliminateBiconditional` mit Type Guards und Rückgabebetyp `Formula1`

Der Ausdruck `typeof f === 'string'` ist ein Type Guard. Ein Type Guard ist in TypeScript eine Laufzeitprüfung, die dem Compiler gleichzeitig als Hinweis dient, einen Union-Typ in einem Kontrollflusszweig auf einen spezifischeren Typ zu verengen. Innerhalb des `if`-Zweigs weiß der Compiler, dass `f` nur noch ein `string` sein kann. Da die Funktion im `if`-Zweig durch ein `return` zwingend vorzeitig verlassen wird, schließt TypeScript den Typ `string` für den weiteren Verlauf des Codes sicher aus. Im impliziten `else`-Zweig bleibt vom ursprünglichen Union-Typ daher nur noch folgender Typ übrig:

```
1 f: ["⊤" | "⊥"] | ["¬", Formula] | ["↔" | "→" | "∧" | "∨", Formula, Formula]
```

Listing 54: Eingeschränkter Union Type

In Zelle 6 wird der erste Operator aus der Formel extrahiert. Dabei wird der Datentyp der Variable `op` auf eine feste Menge von Zeichenketten eingeschränkt. Die Variable `op` besitzt somit den folgenden Typ:

```
1 op: "⊤" | "⊥" | "↔" | "→" | "∧" | "∨" | "¬"
```

Listing 55: Eingeschränkter Union Type von `op`

Durch diese strikte Typisierung kann der TypeScript-Compiler statisch überprüfen und sicherstellen, dass das anschließende `switch`-Statement alle möglichen Fälle vollständig abdeckt.

Während Python auf Pattern Matching mittels `match/case` setzt, um die Baumstruktur direkt über Muster wie (`'↔'`, `g`, `h`) zu zerlegen, verwendet TypeScript eine Kombination aus Type Guards und Type Narrowing. Durch das frühe `return` im `if`-Zweig ist im nachfolgenden Code garantiert, dass `f` kein `string` mehr sein kann. Dies macht die Destructuring-Zuweisung `const [op] = f`; überhaupt erst typsicher möglich. Das anschließende `switch (op)` fungiert als weitere Ebene der Typverengung: Da der Operator `op` auf diskrete `string`-Literal-Typen (wie etwa `'↔'` oder `'¬'`) beschränkt ist, leitet der Compiler für jeden `case` die exakte Array-Struktur von `f` sicher ab.

Dieser wesentliche Unterschied zeigt sich in der statischen Code-Analyse: Das abschließende `return None #type: ignore` in der Python-Variante offenbart, dass der dortige Typchecker die Vollständigkeit der Fallunterscheidung nicht eigenständig beweisen kann. TypeScript nutzt die etablierten Type Guards und die kontrollflussbasierte Verengung hingegen so effektiv, dass der Compiler eine fehlende Fallabdeckung im `switch`-Block sofort erkennen würde. Dadurch wird eine durchgängig höhere Typpräzision gewährleistet.

5.5 Negationsnormalform

Die Funktion `nnf` transformiert eine Formel in *Negationsnormalform* und bildet damit die Schnittstelle zwischen den vorherigen Operator-Eliminierungen und der anschließenden

KNF-Konstruktion. Der zentrale Unterschied der beiden Implementierungen liegt weniger im rekursiven Kontrollfluss, sondern in der typseitigen Spezifikation des Ergebnisses: In TypeScript wird das Ziel explizit durch den eigenständigen Typ `NNF` modelliert, während Python auch nach der Transformation weiterhin den generischen Typ `Formula` verwendet.

```
1  function nnf(f: Formula2): NNF {
2      "Compute the negation normal form of f."
3      if (typeof f === 'string') {
4          return f;
5      }
6      const [op] = f;
7      switch (op) {
8          case 'T':
9              return ['T'];
10         case '⊥':
11             return ['⊥'];
12         case '¬': {
13             const [, g] = f;
14             return neg(g);
15         }
16         case '^':
17         case '∨': {
18             const [, g, h] = f;
19             return [op, nnf(g), nnf(h)];
20         }
21     }
22 }
```

Listing 56: TypeScript: `nnf` mit Eingabetyp `Formula2` und Rückgabetyp `NNF`

Der Typ `NNF` kodiert die strukturelle Invariante „Negation nur vor Variablen“ direkt im Typsystem, da negative Literale ausschließlich als `['¬', Variable]` zugelassen sind und `['¬', NNF]` gerade *nicht* existiert. Damit können nachfolgende Transformationsschritte (insbesondere die **KNF**-Konstruktion) statisch davon ausgehen, dass sie nie einen Negationsknoten über einer zusammengesetzten Teilformel behandeln müssen. Zusätzlich ist der Eingabetyp bereits `Formula2`. Dadurch sind Operatoren wie `'↔'` und `'→'` in `nnf` typseitig ausgeschlossen, was den `switch`-Block auf genau die verbleibenden Fälle reduziert.

```

1  def nnf(f: Formula) -> Formula:
2      'Compute the negation normal form of f.'
3      match f:
4          case str(p):
5              return p
6          case ('T', ) | ('⊥', ):
7              return f
8          case ('¬', g):
9              return neg(g)
10         case (op, g, h):
11             return (op, nnf(g), nnf(h))
12     return None # type: ignore

```

Listing 57: Python: `nnf` mit generischem Rückgabety `Formula`

In Python ist die Funktionalität algorithmisch äquivalent, jedoch bleibt die Rückgabe weiterhin vom Typ `Formula`, sodass die `NNF`-Struktur nicht als Typinvariante festgehalten wird. Deshalb muss dem Typchecker hier wieder mit `#type ignore` unterdrückt werden.

5.6 Transformation in Konjunktive Normalform

Die Funktion `cnf` transformiert eine logische Formel, die sich bereits in `NNF` befindet, in die Konjunktive Normalform. Das Ergebnis wird als Menge von Klauseln repräsentiert, wobei jede Klausel wiederum eine Menge von Literalen darstellt.

```

1  def cnf(f: Formula) -> CNF:
2      match f:
3          case str(p):
4              return { frozenset({p}) }
5          #weitere Fälle ...
6          case ('^', g, h):
7              return cnf(g) | cnf(h)
8          case ('v', g, h):
9              return { k1 | k2 for k1 in cnf(g) for k2 in cnf(h) }
10     return None # type: ignore

```

Listing 58: Python: `cnf` mit Pattern Matching und nativen Sets

```

1  const RS = RecursiveSet
2  function cnf(f: NNF): CNF {
3      if (typeof f == 'string') {
4          return new RS<Clause>(new RS<Literal>(f));
5      }
6      switch (f[0]) {
7          // weitere Fälle ...
8          case '^': {
9              return cnf(f[1]).union(cnf(f[2]));
10         }
11         case 'v': {
12             const Pairs = cnf(f[1]).cartesianProduct(cnf(f[2]));
13             return Pairs.map(([c1, c2]) => c1.union(c2));
14         }
15     }
16 }

```

Listing 59: TypeScript: `cnf` mit Type Guards und `RecursiveSet`

Ähnlich wie bei der Funktion `eliminateBiconditional` zeigen sich hier grundlegende Unterschiede in der Datenmodellierung und im Kontrollfluss. Python nutzt erneut Pattern Matching zur direkten Destrukturierung der Formel. Für Mengenoperationen kommen überladene Operatoren wie `|` für die Vereinigung zum Einsatz. Ein überladener Operator in Python ist ein Symbol, das je nach Datentyp eine unterschiedliche Bedeutung erhält. Während `|` bei Zahlen traditionell als bitweises ODER fungiert, ist es für Set-Objekte so definiert, dass es die Vereinigungsmenge bildet:

```

1  1 | 2          # bitweises ODER (01 | 10), ergibt 3 (11)
2  {1, 2} | {2, 3} # Vereinigungsmenge, ergibt {1, 2, 3}

```

Listing 60: Operator Overloading

In TypeScript wird der Kontrollfluss stattdessen wieder durch einen Type Guard und anschließende kontrollflussbasierte Typanalyse über `switch (f[0])` realisiert. Anstelle der in Python genutzten nativen `set`- und `frozenset`-Strukturen kommt hier die Klasse `RecursiveSet` (abgekürzt als `RS`) zum Einsatz.

Dieser Unterschied im Datenmodell tritt besonders im Fall der Disjunktion deutlich hervor. In Python wird das Distributivgesetz durch das kartesische Produkt der beiden

Klauselmengen und die anschließende paarweise Vereinigung umgesetzt. Dies geschieht äußerst kompakt mittels Set-Comprehension: $\{ k1 \mid k2 \text{ for } k1 \text{ in } \text{cnf}(g) \text{ for } k2 \text{ in } \text{cnf}(h) \}$. Eine Set-Comprehension ermöglicht es über alle möglichen Paare $(k1, k2)$ zu iterieren und deren Vereinigungen direkt als neue Menge zu sammeln.

In TypeScript wird diese Logik explizit über die Methodenaufrufe der `RecursiveSet`-Klasse ausformuliert:

```
1  const Pairs = cnf(f[1]).cartesianProduct(cnf(f[2]));
2  return Pairs.map(([c1, c2]) => c1.union(c2));
```

Zunächst wird durch den Aufruf von `cartesianProduct` das Kreuzprodukt der Klauselmengen `cnf(f[1])` und `cnf(f[2])` gebildet. Das Ergebnis ist eine iterierbare Struktur von Paaren. Anschließend iteriert die `map`-Methode über diese Menge von Paaren und destrukuriert jedes Paar in die beiden Klauseln `c1` und `c2`. Diese werden dann mittels der klasseneigenen `union`-Methode vereinigt.

Während in Python das mathematische Konzept also implizit in seine Sprachkonstrukte eingebettet ist, sind in TypeScript die zugrunde liegenden Mengenoperationen durch einen klaren, funktionalen Methodenketten-Aufruf auf dem `RecursiveSet`-Objekt transparent.

5.7 Triviale Klauseln und Vereinfachung

Die Funktionen `isTrivial` und `simplify` realisieren in beiden Sprachen denselben semantischen Schritt. Triviale Klauseln (d. h. Klauseln, die ein Literal und sein Komplement enthalten) werden erkannt und aus der Klauselmenge entfernt. In der TypeScript-Version wird dies nun konsequent in einer funktionalen Schreibweise formuliert:

```
1  function isTrivial(clause: Clause): boolean {
2      return clause.some(
3          lit => typeof lit === 'string' && clause.has(new Tuple('¬', lit))
4      );
5  }
```

Listing 61: TypeScript: `isTrivial` als Prädikat über `some` (funktionale Schreibweise)

Python verwendet hingegen idiomatisch `any(...)` sowie Set-Comprehensions:

```

1 def isTrivial(Clause: Clause) -> bool:
2     return any(('¬', p) in Clause for p in Clause)

```

Listing 62: Python: `isTrivial` über `any(...)` und Generatorausdruck

Der Ausdruck `clause.some(...)` ist dabei die direkte Entsprechung zu Pythons `any(...)`: Beide brechen frühzeitig ab, sobald ein Element gefunden wurde, welches das Prädikat innerhalb der Klammern erfüllt. Die zusätzliche Bedingung `typeof lit === 'string'` ist in TypeScript erforderlich, weil `Literal` als Union-Typ sowohl positive Literale (Variablen als Strings) als auch negative Literale (Tupelstruktur) zulässt und die Komplementprüfung hier bewusst nur für die positive Richtung $p \mapsto \neg p$ durchgeführt wird. Genau diese einseitige Prüfung ist auch in Python implizit enthalten: Für negative Literale ist `p` kein Variablenname, sondern selbst ein Tupel, weshalb nach `('¬', p)` für jedes Literal `p` gesucht wird.

Die Funktion `simplify` wendet diese Logik anschließend auf die gesamte Klauselmeng an. Sie dient dazu, alle zuvor als trivial erkannten Klauseln herauszufiltern und eine bereinigte Menge zurückzugeben, die ausschließlich aus nicht-trivialen Klauseln besteht.

```

1 function simplify(clauses: CNF): CNF {
2     return clauses.filter(c => !isTrivial(c));
3 }

```

Listing 63: TypeScript: `simplify` durch Filtern der Klauselmenge

```

1 def simplify(Clauses: set[Clause]) -> set[Clause]:
2     return { C for C in Clauses if not isTrivial(C) }

```

Listing 64: Python: `simplify` als Set-Comprehension

Auch `filter` entspricht semantisch der Python-Set-Comprehension. In beiden Fällen wird eine neue Menge konstruiert, die exakt diejenigen Klauseln enthält, für die das Prädikat `not isTrivial(...)` gilt.

5.8 Zusammenfassung der Übersetzungsmuster

Die Funktion `normalize` fasst die zuvor eingeführten Transformationsschritte zu einer linearen Pipeline zusammen und bildet den zentralen Einstiegspunkt für die algorithmische Verarbeitung zur [KNF](#). Da die reine Verkettung der Funktionen sowie die Ausgabetests in beiden Sprachen konzeptionell identisch sind, werden an dieser Stelle abschließend die zentralen Übersetzungsmuster abstrahiert.

Bei der Überführung der Case Study von Python nach TypeScript hat sich gezeigt, dass bestimmte Python-Konstrukte keine direkten Sprachäquivalente in TypeScript besitzen. Diese müssen systematisch durch andere Paradigmen substituiert werden:

- **Pattern Matching (match/case):** Pythons strukturelles Pattern Matching erlaubt die tiefgreifende Dekonstruktion von Bäumen und Tupeln in einem einzigen Schritt. In TypeScript wird dieses Verhalten durch die Kombination aus *Type Guards* (z.B. `typeof f === 'string'`), kontrollflussbasiertem *Type Narrowing* und `switch`-Anweisungen auf diskriminierten Union-Typen nachgebildet. Ergänzendes Destructuring (z.B. `const [op, g, h] = f;`) sorgt dabei für eine vergleichbare Lesbarkeit und bietet den Vorteil einer strikteren statischen Vollständigkeitsprüfung durch den Compiler.
- **Mengen-Komprehensionen (Set-Comprehensions):** Kompakte Konstrukte wie `{ k1 | k2 for k1 in ... }` oder bedingte Mengenbildungen wie `{ C for C in Clauses if ... }` sind in TypeScript syntaktisch nicht vorgesehen. Sie werden systematisch durch funktionale Programmierkonzepte auf iterierbaren Strukturen übersetzt. Dazu zählen insbesondere Methodenketten mit `map`, `filter` und `some` sowie spezialisierte Methodenaufrufe wie `cartesianProduct`.
- **Operator Overloading:** Python ermöglicht die Überladung von Operatoren, wodurch beispielsweise das logische ODER nativ für die Mengenvereinigung genutzt werden kann. Da TypeScript kein Operator Overloading unterstützt, müssen solche Operationen zwingend in benannte, explizite Methodenaufrufe (wie etwa `c1.union(c2)`) übersetzt werden.
- **Wertbasierte Gleichheit und native Sets:** Pythons native Unterstützung für die wertbasierte Gleichheitsprüfung tiefer, unveränderlicher Datenstrukturen

(`frozenset`, `tuple`) existiert im TypeScript-Standard nicht. Da das native TS-`Set` bei Arrays und Objekten lediglich die Speicherreferenz (Reference Equality) vergleicht, erfordert die Übersetzung solcher Strukturen den Einsatz klassenbasierter Hilfsbibliotheken (wie hier `recursive-set`), die Hash- und Gleichheitsfunktionen explizit für baumartige Strukturen implementieren.

Diese abstrakten Muster veranschaulichen, dass die Portierung von Python nach TypeScript selten eine zeilenweise syntaktische Übersetzung darstellt. Vielmehr erfordert sie den bewussten Wechsel hin zu funktionalen Methodenketten und einer wesentlich strikteren, auf Typen fokussierten Datenmodellierung, um die dynamischen und sprachspezifischen Konzepte Pythons typischer abzubilden.

6 Evaluierung und Ergebnisse

In diesem Kapitel wird der Erfolg der Code-Portierung analysiert. Dabei steht insbesondere der Vergleich zwischen der ursprünglichen Python-Implementierung und der neuen TypeScript-Version im Fokus. Ein wesentliches Kriterium hierfür ist die Lesbarkeit und Nachvollziehbarkeit des Codes für Studierende.

6.1 Lesbarkeit und syntaktischer Vergleich

Ein zentrales Ziel der Übersetzung ist es, den Code so zu gestalten, dass dieser didaktisch wertvoll bleibt. Die Lesbarkeit von TypeScript im Vergleich zu Python hängt stark von den Vorkenntnissen der Betrachtenden ab. Für Studierende, die bereits Erfahrungen mit C-basierten Sprachen (wie C, C++ oder Java) gesammelt haben, wirkt die Syntax von TypeScript durch die explizite Typisierung und die Klammerstruktur oft vertrauter. Im Gegensatz dazu besticht Python durch seine minimalistische und pseudocode-artige Syntax, die besonders für absolute Programmieranfänger intuitiver sein kann [1]. Ein weiterer Aspekt ist, dass Python über eine Vielzahl eingebauter Datenstrukturen und Funktionen verfügt, die in TypeScript teilweise manuell nachgebildet oder über externe Bibliotheken eingebunden werden müssen.

6.1.1 Vergleich anhand einfacher Kontrollstrukturen

Zur Veranschaulichung wird zunächst eine einfache Funktion zur Potenzberechnung aus dem Notebook *Power.ipynb* betrachtet.

In der ursprünglichen Python-Version (siehe Listing 65) wird eine einfache `for`-Schleife in Kombination mit der eingebauten `range()`-Funktion verwendet, um die Multiplikation n -mal durchzuführen. Die Syntax ist äußerst kompakt und verzichtet in ihrer Grundform auf Typdeklarationen. Für einen direkteren Vergleich mit TypeScript wurde die Signatur hier jedoch mit Python-Typ-Hinweisen (ähnlich dem *mypy*-Standard) ergänzt.

```
1 def power(m: int, n: int) -> int:
2     r = 1
3     for i in range(n):
4         r *= m
5     return r
```

Listing 65: Python-Beispiel: Potenzberechnung mit Typ-Annotationen

Die äquivalente TypeScript-Implementierung (siehe Listing 66) zeigt die klassische, C-ähnliche `for`-Schleife. Auffällig ist hier die Wahl der Typisierung: Während die Basis `m` und das Ergebnis `r` als `bigint` deklariert sind, um das bei Potenzen rasche Wachstum ohne Präzisionsverlust abzufangen, wird der Exponent `n` als reguläre `number` getypt. Dies ist eine bewusste Designentscheidung, da abgesehen von Basiswerten wie -1 , 0 oder 1 , ein Exponent, der den maximalen Wert des `number`-Typs überschreitet, rechnerisch ohnehin jede verfügbare Speichergrenze sprengen würde.

```
1 function power(m: bigint, n: number): bigint {
2     let r = 1n;
3     for (let i = 0; i < n; i++) {
4         r *= m;
5     }
6     return r;
7 }
```

Listing 66: TypeScript-Beispiel: Potenzberechnung

Beim strukturellen Vergleich fällt auf, dass die Python-Variante kompakter wirkt. Dies liegt primär an der eingebauten `range`-Funktion, die in TypeScript standardmäßig fehlt. Würde man in TypeScript eine vergleichbare `range`-Utility-Funktion implementieren und mit der Array-Methode `reduce` kombinieren, ließe sich die Logik noch deutlich kürzer fassen. Da diese Hilfsfunktion jedoch erst selbst definiert werden müsste, greift die direkte Portierung hier auf die ausführlichere, aber standardkonforme `for`-Schleife zurück, welche für Entwickler mit C-Hintergrund unmittelbar verständlich ist. Beide Versionen bieten durch ihre jeweiligen Typannotationen einen klaren semantischen Mehrwert, der die Fehleranfälligkeit bei der Nutzung reduziert.

6.1.2 Vergleich komplexerer Algorithmen

Die strukturellen Unterschiede werden bei komplexeren Algorithmen noch deutlicher. Als Beispiel dient die Implementierung des [Dijkstra-Algorithmus](#) zur Berechnung kürzester Pfade in [Dijkstra.ipynb](#).

Das Python-Listing 67 nutzt stark die eingebauten Datenstrukturen `dict` (für `Distance`) und `set` (für `Visited`). Der Code ist kompakt und profitiert von Pythons dynamischer Typisierung, was schnelle Prototypisierung ermöglicht, aber zur Laufzeit zu Fehlern führen kann, wenn unerwartete Typen übergeben werden.

```
1  def shortest_path(source, Edges):
2      Distance = { source: 0 }
3      Visited = { source }
4      Fringe = Set()
5      Fringe.insert( (0, source) )
6      while not Fringe.isEmpty():
7          d, u = Fringe.pop() # get and remove smallest element
8          for v, l in Edges[u]:
9              dv = Distance.get(v, None)
10             if dv == None or d + l < dv:
11                 if dv != None:
12                     Fringe.delete( (dv, v) )
13             Distance[v] = d + l
14             Fringe.insert( (d + l, v) )
15             Visited.add(u)
16         return Distance
```

Listing 67: Python-Beispiel: shortest_path

Die TypeScript-Umsetzung in Listing 68 erfordert durch die explizite Typisierung auf den ersten Blick etwas mehr strukturellen Aufwand. Die Signaturen der Übergabeparameter (`Record<string, Array<[string, number]>>`) dokumentieren exakt die erwartete Graphenstruktur. Für die Menge der bereits besuchten Knoten (`visited`) wird der in TypeScript nativ vorhandene Datentyp `Set<string>` verwendet. Dies stellt die direkte und idiomatische Entsprechung zu Pythons `set` dar. Der Algorithmus bleibt somit in seiner Kernlogik sehr nah am Python-Original. Obwohl der syntaktische Schreibaufwand durch die strengen Typdeklarationen im Vergleich zur kompakten Python-Version leicht steigt, zahlt sich dies durch eine signifikant höhere statische Sicherheit aus.

```

1  function shortestPath(
2      source: string,
3      Edges: Record<string, Array<[string, number]>>
4  ): Record<string, number> {
5      const Distance: Record<string, number> = { [source]: 0 };
6      const visited = new Set<string>();
7      const Fringe = new AVLSet<[number, string]>();
8      Fringe.insert([0, source]);
9      while (!Fringe.isEmpty()) {
10         const [d, u] = Fringe.pop();
11         if (visited.has(u)) {
12             continue;
13         }
14         for (const [v, l] of Edges[u]) {
15             const dv = Distance[v];
16             if (dv === undefined || d + 1 < dv) {
17                 if (dv !== undefined) {
18                     Fringe.delete([dv, v]);
19                 }
20                 Distance[v] = d + 1;
21                 Fringe.insert([d + 1, v]);
22             }
23         }
24         visited.add(u);
25     }
26     return Distance;
27 }

```

Listing 68: TypeScript-Beispiel: shortestPath mit nativem Set

6.1.3 Fehlende eingebaute Operatoren und Funktionen

Ein weiterer Faktor, der die Lesbarkeit beeinflusst, ist das Fehlen bestimmter vordefinierter Operatoren in TypeScript, die in Python standardmäßig vorhanden sind. Ein relevantes Beispiel hierfür findet sich im Notebook [06-Davis-Putnam.ipynb](#).

In Python können **Sets** sehr elegant und lesbar mit dem Operator (`|`) vereinigt werden. Dieser Operator fasst die Menge **UsedVars** und ein neues Set, das lediglich das Element **p** enthält, in einer neuen Menge zusammen:

```
1 newUsed = UsedVars | { p }
```

Listing 69: Mengenvereinigung in Python

Da TypeScript kein Operator Overloading (Überladung von Operatoren) für komplexe Objekte unterstützt, muss dieses Verhalten methodisch nachgebildet werden. In der TypeScript-Version wurde dafür eine spezifische Methode `union` in der `RecursiveSet`-Klasse implementiert, welche die Vereinigungs-Funktionalität bereitstellt:

```
1 const nextUsedVars = UsedVars.union(new RecursiveSet<Variable>(p));
```

Listing 70: Mengenvereinigung in TypeScript mittels `.union()`

Dieser Code erfüllt denselben Zweck, ist jedoch syntaktisch strukturierter und erfordert vom Leser das Wissen über die Existenz und Funktionsweise der spezifischen `union`-Methode auf der Klasse `RecursiveSet`.

6.1.4 KI-generierte Code-Komplexität

Bei der Betrachtung der Lesbarkeit muss zudem kritisch betrachtet werden, wie die verwendeten KI-Modelle den Code übersetzt haben. Ein häufig beobachtetes Phänomen war die Generierung von unnötig komplexem oder imperativem Code, wo deklarative Ansätze lesbarer gewesen wären.

Ein Code-Ausschnitt aus dem Notebook [06-Davis-Putnam.ipynb](#) zeigt, wie die KI eine einfache Bedingung zunächst sehr iterativ und fehleranfällig übersetzte:


```
1  let allUnits = true;
2  for (const C of S) {
3      if (C.size !== 1) {
4          allUnits = false;
5          break;
6      }
7  }
8  if (allUnits) {
9      return S;
10 }
```

Listing 71: Imperative (KI-generierte) Überprüfung

Dieser Code iteriert manuell über die Menge S , um zu prüfen, ob alle Elemente C die Größe 1 haben. Diese Logik lässt sich in TypeScript durch die Nutzung von Funktionen höherer Ordnung deutlich kürzer und semantisch klarer formulieren. Erst durch manuelles Refactoring oder den expliziten Hinweis im Prompt, dass die KI wo immer möglich Konzepte der funktionalen Programmierung verwenden soll, entstand die präferierte, deklarative Variante:

```
1  if (S.every(C => C.size === 1)) {
2      return S;
3  }
```

Listing 72: Deklarative (optimierte) Überprüfung in TypeScript

Als Referenz ist hier die ursprüngliche Python-Version von [06-Davis-Putnam](#) zu sehen, die durch die `all()`-Funktion und List Comprehension eine ähnlich hohe Lesbarkeit wie die optimierte TypeScript-Variante aufweist:

```
1  if all(len(C) == 1 for C in S):
2      return S
```

Listing 73: Python-Referenz der Überprüfung

6.2 Laufzeit und Performance

Neben der Lesbarkeit spielt die Ausführungsgeschwindigkeit der übersetzten Code-Basis eine entscheidende Rolle. Dies gilt insbesondere für rechenintensive Such- und Sortialgorithmen, wie sie im Modul *Theoretische Informatik II: Algorithmen und Datenstrukturen* behandelt werden.

6.2.1 Grundsätzliche Performance-Unterschiede

Um die gemessenen Laufzeiten einordnen zu können, muss die zugrunde liegende Architektur der beiden Sprachen betrachtet werden. Python wird in seiner Standard-Implementierung (CPython) zeilenweise zur Laufzeit interpretiert. Dies führt zu einem vergleichsweise hohen Overhead, da jede Instruktion bei der Ausführung in Maschinencode übersetzt werden muss.

TypeScript hingegen wird zunächst in JavaScript kompiliert, welches dann von einer Laufzeitumgebung wie Node.js (basierend auf der V8-Engine) ausgeführt wird. Die V8-Engine nutzt einen sogenannten *Just-in-Time (JIT) Compiler* [10]. Dieser beobachtet den Code während der Ausführung und kompiliert häufig durchlaufene Pfade in hochoptimierten Maschinencode. Zudem hilft die strikte Typisierung von TypeScript indirekt dabei, den Code so zu schreiben, dass die V8-Engine Optimierungen wie *Hidden Classes* besser anwenden kann. Folglich ist Node.js bei reinen Berechnungsaufgaben, die nicht auf externe, in C/C++ geschriebene Bibliotheken wie NumPy ausgelagert werden, in der Regel deutlich schneller als reines Python.

6.2.2 Laufzeitvergleich: Merge-Sort

Um diese theoretischen Grundlagen in der Praxis zu überprüfen, wurden Laufzeitmessungen an den portierten Algorithmen vorgenommen. Alle nachfolgenden Tests wurden auf einem Microsoft Surface 8 mit einem *11th Gen Intel(R) Core(TM) i5-1135G7 @ 2.40GHz* durchgeführt.

Als exemplarisches Testobjekt dient die Implementierung des Merge-Sort-Algorithmus *Merge-Sort.ipynb*. Da sich der übersetzte TypeScript-Code in seiner Struktur und Logik kaum vom Python-Original unterscheidet, ist ein direkter Performance-Vergleich aussagekräftig.

Für die Messung in Python wurde die integrierte Jupyter-Notebook-*Magic-Command* `%%time` verwendet:

```
1 %%time
2 testSort(100, 2000)
```

Listing 74: Time-Funktion in Python

Ausgabe (Python):

```
All tests successful!
CPU times: total: 8.91 s
Wall time: 18.2 s
```

Für die korrekte Interpretation der Python-Ausgabe muss zwischen CPU time, der reinen Rechenzeit der Prozessorkerne, und Wall time, der real verstrichenen Zeit vom Start bis zum Ende der Zelle, unterschieden werden [9]. Maßgeblich für das Empfinden des Nutzers ist die Wall time von 18,2 Sekunden.

Die Messung in der TypeScript-Version erfolgte mithilfe der nativen `console.time()`-API. Sie misst die real verstrichene Zeit und entspricht damit der Wall time in Python:

```
1 console.time();
2 testSort(100, 2000);
3 console.timeEnd();
```

Ausgabe (TypeScript):

```
All tests successful!
default: 10.618s
```

6.2.3 Analyse der Ergebnisse (Merge-Sort)

Der Vergleich zeigt einen signifikanten Performance-Vorteil der TypeScript-Implementierung. Mit einer Laufzeit von rund 10,6 Sekunden ist die Ausführung in Node.js/TypeScript etwa 42 % schneller als die Python-Variante mit 18,2 Sekunden.

Dieser deutliche Unterschied lässt sich direkt auf die JIT-Kompilierung der V8-Engine zurückführen. Bei einem rekursiven Sortieralgorithmus wie Merge-Sort werden dieselben Funktionen und Code-Pfade tausendfach mit ähnlichen Datentypen aufgerufen. Während der Python-Interpreter diese wiederholten Aufrufe immer wieder neu verarbeiten muss, erkennt die V8-Engine diese Muster schnell und generiert optimierten Maschinencode. Dieser Performance-Gewinn ist besonders im Kontext von Lehrmaterialien vorteilhaft, da Studierende bei der Ausführung komplexer Testfälle in den Notebooks weniger Wartezeiten in Kauf nehmen müssen.

6.2.4 8-Damen-Problem

Das *Davis–Putnam-Verfahren* ist ein klassischer Algorithmus zur Lösung von **SAT-Problemen**, d. h. zur Entscheidung, ob eine aussagenlogische Formel in **KNF** erfüllbar ist. Die Eingabe besteht dabei aus einer Menge von Klauseln. Der Algorithmus arbeitet in zwei Phasen: Zunächst wird in der *Unit-Propagation* (**saturate**) jede Klausel, die nur ein einziges Literal enthält, sofort ausgewertet. Das Literal wird auf wahr gesetzt und alle betroffenen Klauseln werden entsprechend vereinfacht (**reduce**). Anschließend wählt der Algorithmus gesteuert durch eine Heuristik ein noch nicht belegtes Literal, setzt es probeweise auf wahr und ruft sich rekursiv auf. Führt ein Ast zu einem Widerspruch, wird zurückgesetzt und der andere Wahrheitswert versucht. Die *Jeroslow–Wang-Heuristik* priorisiert dabei das Literal, das in möglichst vielen kurzen Klauseln vorkommt, um den Suchraum so früh wie möglich einzuschränken.

Das **8-Damen-Problem** wird in beiden Sprachen zunächst als SAT-Problem in **KNF** kodiert und anschließend mit dem Davis-Putnam-Verfahren gelöst. Da $n = 8$ in der Praxis sehr schnell lösbar ist, wurde für eine aussagekräftigere Messung $n = 16$ gewählt, da hier die Heuristik und die zugrunde liegenden Datenstrukturen deutlich stärker beansprucht werden.

Der Davis–Putnam-Code erzeugt und verarbeitet in kurzer Zeit enorm viele temporäre Klauselmengen und Unit-Klauseln. In beiden Implementierungen sind genau diese ständigen Mengenoperationen und Duplikat-Prüfungen der dominierende Kostenfaktor, insbesondere in den Kernfunktionen **saturate** und **reduce**. Um diesen Architekturunterschied zu veranschaulichen, betrachten wir die **reduce**-Funktion in beiden Sprachen. Diese Funktion vereinfacht die Klauselmenge unter der Annahme, dass ein bestimmtes Literal l wahr

ist.

```
1 def reduce(Clauses: set[Clause], l: Literal) -> set[Clause]:
2     lBar = complement(l)
3     return { C - { lBar } for C in Clauses if lBar in C } \
4         | { C for C in Clauses if lBar not in C and l not in C } \
5         | { frozenset({l}) }
```

Listing 75: Reduktionsschritt in Python mit Set-Comprehensions

In Python wird die Logik extrem prägnant mit **frozenset** (Klausel) und **set** (Klauselmengen) umgesetzt. Der wesentliche Performance-Impact liegt hier in der intensiven Speicherallokation und redundanten Iterationen, die durch die kompakte Syntax verdeckt werden: Für die Reduktion muss der Interpreter drei separate Set-Comprehensions auswerten, was bedeutet, dass unter der Haube mehrfach über **Clauses** iteriert wird. Besonders teuer ist das ständige Erzeugen temporärer Objekte: Der Ausdruck $C - \{ lBar \}$ zwingt Python dazu, in jedem Schleifendurchlauf ein neues Set für die Differenzbildung zu allokalieren. Zudem generiert jede Comprehension eine eigene temporäre Mengen-Instanz, bevor der Vereinigungsoperator (**|**) diese Zwischenergebnisse in ein weiteres, komplett neues **set** zusammenkopiert. Dieser Overhead summiert sich bei tausenden Aufrufen in der Rekursion massiv auf.

In TypeScript wurde dieselbe algorithmische Idee durch den Basis-Prompt Prompt zuerst deutlich imperativer abgebildet. Der Code greift auf die **recursive-set**-Bibliothek zurück, da TypeScript native Set-Comprehensions für tiefgreifende Wertvergleiche nicht unterstützt:

```

1  function reduce(Clauses: Clauses, l: Literal): Clauses {
2      const lBar = complement(l);
3      const result = new RecursiveSet<Clause>();
4      for (const clause of Clauses) {
5          let hasL = false, hasLBar = false;
6          for (const lit of clause) {
7              if (sameLiteral(lit, l)) hasL = true;
8              if (sameLiteral(lit, lBar)) hasLBar = true;
9          }
10         if (hasLBar) {
11             const newClause = new RecursiveSet<Literal>();
12             for (const lit of clause) {
13                 if (!sameLiteral(lit, lBar)) newClause.add(lit);
14             }
15             result.add(newClause);
16         } else if (!hasL) {
17             result.add(clause);
18         }
19     }
20     result.add(new RecursiveSet<Literal>(l));
21     return result;
22 }

```

Listing 76: Reduktionsschritt in TypeScript mit RecursiveSet

Der entscheidende Performance-Vorteil dieser textlastigeren Schreibweise liegt in der strikten Vermeidung unnötiger Allokationen. Anstatt temporäre Zwischenmen- gen zu erzeugen, wird initial genau eine einzige Zielmenge (`const result = new RecursiveSet<Clause>();`) allokiert, und es wird nur ein einziges Mal über `Clauses` iteriert. Eine neue Klausel (`new RecursiveSet<Literal>()`) wird im Speicher aus- schließlich dann instanziiert, wenn das Literal `lBar` tatsächlich entfernt werden muss (`if (hasLBar)`). Bleibt eine Klausel von der Reduktion unberührt (`else if (!hasL)`), wird lediglich ihre bestehende Speicherreferenz in das neue Set (`result.add(clause)`) übernommen wodurch ein teurer Kopiervorgang völlig entfällt.

Wenn man die KI jedoch dazu auffordert, deutlich deklarativeren Code zu generieren, ähnelt die Schreibweise und Länge des Codes der Python-Version aus [6.2.4](#):

```

1  function reduce(Clauses: Clauses, l: Literal): Clauses {
2      const lBar = complement(l);
3      const singletonLBar = single(lBar);
4      const result = Clauses.filterMap(
5          (clause) => !clause.has(l),
6          (clause) => clause.has(lBar) ? clause.difference(singletonLBar) : clause
7      );
8      result.add(new RecursiveSet<Literal>(l));
9      return result;
10 }

```

Listing 77: Reduktionsschritt in TypeScript mit RecursiveSet

Ein zentraler Beschleuniger in der TypeScript-Version ist dabei die `recursive-set`-Datenstruktur selbst. Sie implementiert Hash-Tabellen mit [Open Addressing](#) und [Linear Probing](#) sowie eine auf Performance ausgelegte Hash-Funktion. Grundlegende Operationen wie `add/has` sind damit amortisiert $\mathcal{O}(1)$.

Die im `recursive-set`-Modul formulierten Voraussetzungen (keine NaN/Infinity, keine plain objects, gute Hash-Verteilung, Unveränderlichkeit des Hashcodes nach dem Einfügen und keine Zyklen) sind in diesem SAT-Use-Case voll erfüllt, da Literale und Variablen als Strings bzw. Tuple-Objekte modelliert und die erzeugten Klauseln nach der Konstruktion nicht mehr mutiert werden. Damit greifen die effizienten `add`-Operationen im TypeScript-Code extrem schnell, was den imperativen Ansatz für das ständige Erzeugen und Prüfen von Unit-Klauseln in der Gesamtlaufzeit überlegen macht.

6.2.5 Laufzeitvergleich: Davis–Putnam mit JW-Heuristik ($n = 16$)

Analog zum Merge-Sort-Vergleich wurden die gleichen Methoden zum Messen der Zeit verwendet.

```

1  %%time
2  Solution = queens(16)

```

Listing 78: Zeitmessung in Python (16-Damen)

Ausgabe (Python):

CPU times: total: 23.6 s

Wall time: 24.8 s

```
1 console.time("queens");  
2 const Solution = queens(16);  
3 console.timeEnd("queens");
```

Listing 79: Zeitmessung in TypeScript (16-Damen)

Ausgabe (TypeScript):

queens: 17.041s

6.2.6 Analyse der Ergebnisse (8/16-Damen)

Für $n = 16$ zeigt sich ein deutlicher Performance-Vorteil der TypeScript/Node.js-Ausführung: 17,041 s gegenüber 24,8 s in Python, also rund 31 % weniger *Wall time*.

Gerade beim Davis–Putnam-Verfahren treten wiederkehrende Iterationsmuster auf (z. B. in der Iteration durch Klauseln in **reduce**). Dadurch kann die V8-Engine in diesen Pfaden durch JIT-Kompilierung deutlich stärker profitieren als eine rein bytecode-interpretierte Ausführung in Python.

Im Vergleich zum Merge-Sort-Benchmark fällt der relative Performance-Abstand beim n -Damen-Problem jedoch spürbar geringer aus. Wie zuvor beschrieben, verlagern sich die Kosten bei Davis–Putnam stark in Richtung Speicherverwaltung (Aufrufe wie **add/has/union** sowie Objekterstellung). Die Laufzeit wird also weniger von engen numerischen Rechenschleifen bestimmt, in denen JIT-Compiler meist am stärksten glänzen. Hier kommt Python ein entscheidender Architekturvorteil zugute: Bei diesem speicherintensiven Workload führt Python nicht nur reines Python aus, sondern profitiert massiv von seinen hochoptimierten, in C implementierten Hash-Tabellen für **set** und **frozenset**. Insbesondere sind zentrale Operationen wie Membership-Checks und das Hashing von **frozenset** in CPython direkt in C realisiert. Zudem wird der Hashwert eines **frozenset** nach der ersten Berechnung intern gecacht, was den Aufwand für wiederholte Hash-Berechnungen drastisch reduziert und den Performance-Nachteil des reinen Interpreters bei diesem spezifischen Problem stark abfedert.

7 Fazit und Ausblick

Die vorliegende Studienarbeit hatte zum Ziel, das bestehende Lehrmaterial für die Vorlesungen der Theoretischen Informatik und Algorithmen und Datenstrukturen von Python nach TypeScript zu portieren. Nach Abschluss der Projektphase lässt sich festhalten, dass die Kernziele erfolgreich erreicht wurden, wenngleich der Prozess sowohl inhaltliche als auch technische Herausforderungen offenbarte.

7.1 Allgemeine Bewertung des Übersetzungsprozesses

Insgesamt wurden annähernd 80 Jupyter Notebooks erfolgreich in TypeScript übersetzt. Mit Ausnahme des Notebooks [11-Prover.ipynb](#), was mit dem Dozenten abgesprochen wurde, sind alle Dokumente funktional korrekt, lassen sich im TypeScript-Kernel ausführen und können in der aktuellen Form in Vorlesungen eingesetzt werden.

Der initiale Aufwand für die Konvertierung erwies sich jedoch als immens und nahm die gesamte Kapazität eines Studiensemesters in Anspruch. Dieser Aufwand resultierte nicht nur aus der reinen Menge an Dokumenten, sondern vor allem aus der Notwendigkeit, sich tiefgehend in neue Bibliotheken, die TypeScript-spezifische Typisierung und die fachlichen Details der teilweise hochkomplexen Algorithmen einzuarbeiten. Zudem unterlag der Projektrahmen folgender Dynamik: Änderungen an den Anforderungen während und nach der initialen Übersetzungsphase erforderten eine kontinuierliche Anpassung und Wartung der Notebooks.

Daraus resultiert auch der aktuelle Stand der Dokumente: Während die grundlegende Funktionalität gegeben ist, besteht weiterhin Optimierungspotenzial. Dies zeigt sich insbesondere im Modul Logik, wo *Chapter 5* aufgrund des massiven Zeitaufwands für das vorhergehende Kapitel noch nicht den gleichen Präzisionsgrad aufweist wie *Chapter 4*. An vielen Stellen ließe sich der generierte Code, teils Resultat der noch imperfekten KI-Unterstützung, weiter vereinfachen und refaktorisieren. Angesichts des auf sechs Monate begrenzten Zeitrahmens war eine lückenlose, tiefgreifende Optimierung jedes einzelnen Notebooks jedoch nicht realisierbar.

7.2 Lesbarkeit und Didaktik

Ein zentraler Aspekt von Lehrmaterialien ist deren Verständlichkeit für Studierende. Die Einführung der strikten Typisierung in TypeScript macht den Code im Vergleich zu Python auf den ersten Blick syntaktisch anspruchsvoller. Dies erfordert von den Studierenden eine gewisse Einarbeitungszeit in die Typ-Syntax.

Die Bewertung, ob TypeScript schwerer lesbar ist als Python, unterliegt jedoch teilweise einer subjektiven Verzerrung, da unser Wissen in diesen Fächern ursprünglich anhand der kompakten Python-Syntax erworben wurde. Objektiv betrachtet gilt TypeScript als eine sehr gut strukturierte und lesbare Sprache. Eine verringerte Lesbarkeit in den aktuellen Notebooks ist oft weniger der Sprache an sich geschuldet, sondern vielmehr der Tatsache, dass Code-Qualität stark von der Erfahrung der Entwickler abhängt. Da die Übersetzung aus der Perspektive von Junior-Entwicklern in TypeScript erfolgte, besteht hier noch Spielraum für eine elegantere und idiomatischere Gestaltung des Codes. Dennoch bietet das statische Typsystem einen massiven didaktischen Vorteil: Es zwingt die Studierenden, sich explizit mit den Datentypen und Signaturen von Algorithmen auseinanderzusetzen, was zu einem deutlich sichereren Code und einem tieferen Verständnis der Datenstrukturen führt. Die endgültige Evaluation der didaktischen Eignung wird jedoch erst durch das direkte Feedback der Studierenden in kommenden Semestern möglich sein, wobei deren programmiertechnische Vorerfahrungen eine entscheidende Rolle spielen werden.

7.3 Performance-Betrachtung

Im Bereich der Laufzeit zeigte die Portierung nach TypeScript größtenteils positive Ergebnisse. Insbesondere bei rechenintensiven Algorithmen, die stark auf `for`-Schleifen und klassische mathematische Berechnungen setzen (wie etwa Sortieralgorithmen), übertrifft die Ausführungsgeschwindigkeit durch die JIT-Kompilierung der V8-Engine die interpretierte Python-Version deutlich.

Bei komplexeren, abstrakten Datenstrukturen zeigte sich ein differenzierteres Bild. Algorithmen, die intensiv Gebrauch von Sets oder der eigens implementierten `RecursiveSet`-Struktur machen, profitierten weniger stark von der Node.js-Umgebung. Hier waren die Performance-Gewinne gegenüber Python zwar messbar, fielen jedoch geringer aus als bei rein iterativen Berechnungen.

7.4 Ausblick

Die portierten Notebooks bilden ein solides Fundament für die zukünftige Lehre in TypeScript. Für Folgeprojekte oder künftige Überarbeitungen wird empfohlen, den Code schrittweise weiter zu vereinfachen und an TypeScript-spezifische Design-Pattern anzupassen. Sobald das Material im regulären Vorlesungsbetrieb eingesetzt wird, wird eine gezielte Evaluation durch die Studierenden essenziell sein, um die Balance zwischen Typ-Sicherheit und didaktischer Komplexität weiter zu optimieren.

Literaturverzeichnis

- [1] Linda Mannila und Michael De Raadt. *An objective comparison of languages for teaching introductory programming*. 2006. URL: https://www.academia.edu/16624729/An_objective_comparison_of_languages_for_teaching_introductory_programming (besucht am 01.03.2026).
- [2] Gavin Bierman, Martín Abadi und Mads Torgersen. *Understanding TypeScript*. 2014. URL: https://link.springer.com/chapter/10.1007/978-3-662-44202-9_11 (besucht am 19.01.2026).
- [3] Lorena A. Barba u. a. *Teaching and Learning with Jupyter*. URL: <https://jupyter4edu.github.io/jupyter-edu-book/> (besucht am 19.01.2026).
- [4] Microsoft Corporation. *TypeScript Documentation*. URL: <https://www.typescriptlang.org/docs/> (besucht am 25.01.2026).
- [5] Microsoft Corporation. *TypeScript Repository*. URL: <https://github.com/Microsoft/TypeScript?tab=readme-ov-file> (besucht am 25.01.2026).
- [6] Stack Overflow Community. *What is Monkey Patching?* URL: <https://stackoverflow.com/questions/5626193/what-is-monkey-patching> (besucht am 15.02.2026).
- [7] Karl Stroetmann. *Algorithms Repository*. URL: <https://github.com/karlstroetmann/Algorithms> (besucht am 13.10.2025).
- [8] Karl Stroetmann. *Logic Repository*. URL: <https://github.com/karlstroetmann/Logic> (besucht am 13.10.2025).
- [9] The IPython Development Team. *Built-in magic commands: %%time*. URL: <https://ipython.readthedocs.io/en/stable/interactive/magics.html#magic-time> (besucht am 01.03.2026).
- [10] V8 Project Authors. *V8 JavaScript engine*. URL: <https://v8.dev/docs> (besucht am 01.03.2026).
- [11] Wikipedia-Autoren. *TypeScript*. URL: <https://en.wikipedia.org/wiki/TypeScript> (besucht am 25.01.2026).